

Corso Godot

Super-guida · tutto il materiale in un unico documento · v1.3 · 16/08/2026

Sommario

Corso Godot · panoramica generale (doc 00)	2
Glossario (doc 01)	6
Manuale · libro di testo · v0.20	9
Eserciziario · v0.15	43
Quaderno dello studente · modello	66
Scaletta delle lezioni (doc 03)	68
Struttura del repository (doc 04)	71
Consegne · come sono organizzate	74
Guida · crearsi l'account GitHub	76
Guida · invito alla classe, primo giro	78
Modello consegna · istruzioni	81
Modello consegna · scheda	83
Prof · come correggere	84
Prof · modello di valutazione	86
Esempio consegna · scheda	87
Esempio consegna · valutazione	88
Esercizi pronti · panoramica	89
Gioco · Affonda la Bonomi	90
Gioco · Gioco del Quindici	93
Regole di formattazione dei documenti	95
README · porta d'ingresso del repository	99
Riprendiamo da qui · nota di lavoro tra sessioni	101
02 INDICE	107

00 Corso Godot — tutto quello che abbiamo creato

Versione 1.1 — 16/08/2026

Quadro completo del corso: manuale, esercizi, giochi, documenti e strumenti. Per l'elenco sempre aggiornato dei percorsi vedi anche il documento 02 Indice.

1. Il manuale, libro di testo, versione 0.20

1. Teoria di Godot spiegata ai ragazzi, con il ponte da Lazarus, il sommario cliccabile e le foto dell'ambiente e del gioco di ogni esercizio. Sorgente `manuale/manuale.md` , PDF `manuale-v0.20.pdf` .
2. Schede iniziali:
3. Scheda 1, come si valutano i compiti.
4. Scheda 2, scrivere in Markdown (MD: il modo semplice di scrivere i testi).
5. Scheda 3, la tua copia del corso e come consegnare.
6. Capitoli:
7. Capitolo 0, cos'è Godot, il parente di Lazarus.
8. Capitolo 1, i 4 concetti base di Godot.
9. Capitolo 2, GDScript, il linguaggio.
10. Capitolo 3, i mattoncini, uno alla volta, 11 micro-lezioni a scoperta graduale.
11. Capitolo 4, costruiamo l'Esercizio 1, il bottone che saluta.
12. Capitolo 5, costruiamo l'Esercizio 2, muovi il quadrato.
13. Capitolo 6, costruiamo l'Esercizio 3, prendi la moneta.
14. Capitolo 7, costruiamo l'Esercizio 4, acchiappa le stelle.
15. Capitolo 8, il percorso, dagli esercizi al progetto boss.

2. L'eserciziario, versione 0.15

1. Gli esercizi per i ragazzi, ognuno con i 4 livelli di aiuto a scoperta graduale: descrizione, aiuto, la scena, codice completo. Sorgente `manuale/eserciziario.md` , PDF `eserciziario-v0.15.pdf` .
2. Elenco degli esercizi:
3. Esercizio 1, il bottone che saluta.
4. Esercizio 2, muovi il quadrato.

5. Esercizio 3, prendi la moneta.
6. Esercizio 4, acchiappa le stelle.
7. Esercizio boss, Affonda la Bonomi.
8. C'è anche il quaderno dello studente, il portfolio personale che ogni ragazzo fa crescere: `manuale/quaderno-studente-TEMPLATE.md`.

3. Gli esercizi pronti, progetti Godot

1. Progetti funzionanti, uno per cartella, tutti avviati e verificati senza errori. Convenzione dei nomi: la variabile finisce in Var, il nodo nella scena finisce in Scena.
2. Cartelle:
3. `esercizi/01-bottone-che-saluta/`.
4. `esercizi/02-muovi-il-quadrato/`.
5. `esercizi/03-prendi-la-moneta/`.
6. `esercizi/04-acchiappa-le-stelle/`.

4. I giochi, giocabili dal browser anche da telefono

1. Aprili e giochi, senza installare niente. I quattro giochi arcade (talpa, asteroidi, mattoni, torta) hanno i suoni e una classifica online condivisa dei primi 10: a fine partita si scrive il nome con una tastiera a bottoni e si sfidano tutti gli altri giocatori.
2. Indirizzi dei giochi:

3. Affonda la Bonomi, battaglia navale in 3D.

<https://nicolaregge-pulse.github.io/corso-godot/>

2. Gioco del Quindici, con la foto incorporata e le tessere di legno.

<https://nicolaregge-pulse.github.io/corso-godot/quindici/>

3. Acchiappa la talpa, le talpe spuntano dalla terra e le tocchi a tempo.

<https://nicolaregge-pulse.github.io/corso-godot/talpa/>

4. Schiva gli asteroidi, muovi la navetta a triangolo e sopravvivi.

<https://nicolaregge-pulse.github.io/corso-godot/asteroidi/>

5. Rompi i mattoni, racchetta e pallina che spaccano il muro.

<https://nicolaregge-pulse.github.io/corso-godot/mattoni/>

6. Torta in faccia, tiri col dito e centri il bersaglio sulla faccia.

<https://nicolaregge-pulse.github.io/corso-godot/torta/>

3. I sorgenti Godot sono nelle cartelle omonime in radice. Ogni gioco ha le costanti FALLO TUO in cima al codice, per la personalizzazione.

5. La cassaforte

1. App che chiude un file con una password, tutto sul dispositivo, niente in rete. Online:

<https://nicolaregge-pulse.github.io/corso-godot/cassaforte/>

6. Il kit di consegna dei ragazzi, cartella consegne

1. Tutto per raccogliere e correggere i lavori, con guide in MD e in PDF.

2. Contenuto:

3. Crea account GitHub, come i ragazzi si fanno l'account.

4. Invito alla classe, la guida del primo giro, dal fork alla prima consegna.

5. Modello, il kit vuoto che i ragazzi copiano, con scheda e istruzioni.

6. Prof, solo per il docente: come correggere, i quattro segnali e la scala del voto.

7. Un esempio già compilato in

[consegne/2026-2027-1informatica/rossi-mario/es1-bottone/](https://nicolaregge-pulse.github.io/consegne/2026-2027-1informatica/rossi-mario/es1-bottone/).

8. Regola presa: le attività coi ragazzi, account e primo giro, si fanno in classe a settembre, non prima.

7. Guide, note e strumenti

1. Documento 03 Scaletta lezioni, il piano delle prime lezioni, lezione per lezione.
2. Documento 02 Indice, la mappa sempre aggiornata di tutti i percorsi.
3. Documento 01 Glossario, i termini e le sigle spiegati.
4. Nota di servizio interna RIPRENDIAMO-DA-QUI, dove siamo e cosa manca tra una sessione e l'altra.
5. Cartella `manuale/_build/`, gli strumenti che generano i PDF: `genera_pdf` per manuale ed eserciziario, `guida_pdf` per le guide e i documenti, `volume_pdf` per la super-guida combinata.
6. Tutti i documenti del corso sono disponibili sia in Markdown sia in PDF.

8. Come è organizzato e versionato

1. Il ramo `main` è l'area di lavoro del prof, sempre aggiornata.
2. Le release taggate, come v1.0 e v1.1, sono le versioni congelate e stabili per i ragazzi.
3. I giochi web si pubblicano da soli su GitHub Pages a ogni modifica, dalla cartella docs.
4. I ragazzi lavorano su una loro copia, il fork, mai sul repository del prof.

01 Glossario

Versione 1.0 — 16/08/2026

Termini e sigle del corso, spiegati in parole semplici. Alla prima volta che un documento usa una sigla, la scrive per esteso; qui trovi la spiegazione completa.

1. Godot e la programmazione

Termine	Spiegazione
Godot	Il motore di gioco (game engine) che usiamo per costruire i giochi. È portabile: un solo file, senza installazione.
GDScript	Il linguaggio di programmazione di Godot. Assomiglia molto a Python: si va a capo e si indenta, senza punto e virgola.
Nodo (Node)	Il mattoncino base di Godot. Ne esistono di tanti tipi. Come un pezzo di LEGO.
Scena (Scene)	Un albero di nodi salvato in un file con estensione tscn. Un livello, un menu, un personaggio. Come una costruzione di LEGO.
Script	Il codice GDScript attaccato a un nodo, che gli dà un comportamento.
Segnale (Signal)	Un annuncio che un nodo manda quando succede qualcosa, per esempio il click di un bottone.
Game loop	Il battito continuo del gioco: la funzione <code>_process(delta)</code> gira da sola circa 60 volte al secondo.
Proprietà	Una caratteristica di un nodo, per esempio il testo di una Label o il colore di un rettangolo.
Ispettore	Il pannello di Godot dove si cambiano le proprietà del nodo selezionato.
Esportazione (export)	Creare la versione del gioco che gira nel browser (la versione web).

2. Tipi di nodo più usati

Termine	Spiegazione
Node2D	Un contenitore per il 2D: dentro ci si mettono gli altri nodi del gioco.
Sprite2D	Un nodo che mostra un'immagine.
Label	Un nodo che mostra del testo.

Termine	Spiegazione
Button	Un bottone da premere.
LineEdit	Una casella dove scrivere del testo.
ColorRect	Un rettangolo colorato.
AudioStreamPlayer	Il nodo che suona un effetto sonoro.

3. Il ponte da Lazarus

Termine	Spiegazione
Lazarus	L'ambiente che i ragazzi già conoscono, con Free Pascal. Da lì partiamo per spiegare Godot.
Form	In Lazarus la finestra del programma. In Godot diventa la Scena.
Componente	In Lazarus il pezzo pronto, per esempio TButton o TEdit. In Godot diventa il Nodo.
Caption	In Lazarus il testo mostrato da un componente. In Godot è la proprietà del testo, per esempio il text di una Label.
Gestore evento	In Lazarus la procedura del click, per esempio Button1Click. In Godot è un segnale collegato a una funzione.

4. Git, GitHub e la pubblicazione

Termine	Spiegazione
Git	Il sistema di versionamento: tiene la storia di tutte le modifiche.
GitHub	Il sito dove teniamo online il corso e i giochi.
Repository (repo)	La cartella del progetto tenuta su Git e GitHub.
Commit	Un salvataggio, con un messaggio che dice cosa è cambiato.
Branch (ramo)	Una linea di lavoro separata, per non disturbare il lavoro degli altri.
PR (Pull Request: richiesta di unione)	La proposta di unire un ramo nel lavoro principale.
Merge	L'unione di due lavori.
Fork	La propria copia personale di un repository.

Termine	Spiegazione
Release	Una versione congelata e pubblicata, stabile per i ragazzi, per esempio v1.0.
GitHub Pages	Il servizio di GitHub che pubblica i giochi sul web, dalla cartella docs.

5. La classifica online dei giochi

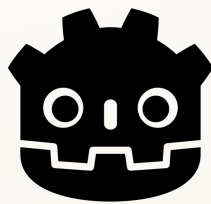
Termine	Spiegazione
Firebase Realtime Database	Il database online, sui server di Google, dove finiscono i punteggi della classifica condivisa.
REST (REpresentational State Transfer)	Il modo con cui i giochi, dal browser, leggono e scrivono i punteggi nel database via internet.
Cache	La memoria del browser che a volte tiene la versione vecchia di una pagina. Per questo i link dei giochi sono fatti per aggiornarsi da soli.

6. I documenti del corso

Termine	Spiegazione
Markdown (MD)	Il modo semplice di scrivere i documenti, con pochi segni. È il sorgente da cui si genera il PDF.
PDF (Portable Document Format: formato del documento portabile)	Il formato del documento stampabile, generato dall'MD.
Fase 1	La prima parte del corso: esercizi separati, con un commit per salvare.
Fase 2	La seconda parte: un progetto di gruppo che cresce, con branch, Pull Request e release.

Changelog

Versione	Data	Cosa è cambiato
1.0	16/08/2026	Prima stesura del glossario del corso.



GODOT
Game engine

Il Manuale

Corso di Godot

Corso a cura del prof. Nicola Regge

Sommario

Perché quello che impari qui può cambiarti le cose

SCHEDA 1 Come si valutano i compiti

SCHEDA 2 Scrivere in Markdown

SCHEDA 3 La tua copia del corso, e come consegnare

CAPITOLO 0 Cos'è Godot, il parente di Lazarus

CAPITOLO 1 I 4 concetti base di Godot

CAPITOLO 2 GDScript: il linguaggio

CAPITOLO 3 I mattoncini, uno alla volta

CAPITOLO 4 Costruiamo l'Esercizio 1: il bottone che saluta

CAPITOLO 5 Costruiamo l'Esercizio 2: muovi il quadrato

CAPITOLO 6 Costruiamo l'Esercizio 3: prendi la moneta

CAPITOLO 7 Costruiamo l'Esercizio 4: acchiappa le stelle

CAPITOLO 8 Il percorso: dagli esercizi al “progetto boss”

Come useremo l'AI

Perché quello che impari qui può cambiarti le cose

Due parole mie, prima di cominciare.

*Voglio dirti una cosa, dritta, prima di tutto il resto. Alla tua età **nessuno si aspetta che tu sia già un esperto**. Chi ti darà il primo tirocinio, il primo lavoro, non guarderà per prima cosa quanto codice sai scrivere: quello te lo insegnano. Sceglierà **te**, e non un altro, soprattutto per **due cose**.*

***La prima: come ti poni.** Se ci sei ogni mattina, in orario. Se **rispondi** quando qualcuno ti scrive. Se, quando sbagli, hai il coraggio di dire “ho sbagliato, lo aggiusto” invece di nasconderti. Se **non molli** al primo “no”. Se sai stare in squadra. Non sono cose “da grandi”: sono cose che **puoi decidere di fare da domani**, e per me pesano più di mille nozioni.*

***La seconda: cosa sai fare e mostrare.** Non parole — roba fatta **con le tue mani**. Un gioco che gira. Una cosa che apri sul telefono e dici, guardando negli occhi chi hai davanti: “**Questo l’ho fatto io.**”*

*Queste due cose **insieme** fanno scegliere te. Per questo non ti faccio fare esercizi da scuola e basta: ogni cosa che costruisci, anche piccola, va nel **tuo** quaderno. Pezzo dopo pezzo ti costruisci una prova — che **sai fare**, e che su di te si può contare.*

*E adesso la cosa che mi sta più a cuore. Quest’anno lo passiamo **insieme**: ci scherzeremo, giocheremo, cresceremo. Certi giorni ci abbracceremo, altri litigheremo — ci mostreremo pure i denti. Ti spingerò forte, ti farò arrabbiare, ti darò voti che non ti piacciono, e sì, potrei perfino doverti bocciare. Ma **qualunque cosa succeda, sono dalla tua parte**: se sono duro non è contro di te, è perché **credo in te** e voglio il meglio che puoi diventare.*

*C’è un patto, però. **In laboratorio siamo professionisti**, con rispetto dei ruoli: io il tuo **formatore**, tu il mio **allievo**. Fuori, al bar, è un’altra musica: ci scappa la battuta, ci prendiamo pure in giro — ma sempre con rispetto. E **davanti agli altri** siamo sempre educati: chi ci vede non sa che rapporto abbiamo, e la prima impressione parla di **tutti e due**.*

*Molti di voi partono da lontano, da porte che si sono chiuse. Questa è una porta che **si apre**, e te la apro io. Non sprecarla: prenditi ogni piccola vittoria. Il resto — il lavoro, il rispetto, le occasioni — nasce da lì. Ci credo. E **credo in te**.*

Nicola

Come si valutano i compiti

Le regole del gioco, chiare fin da subito.

Qui non ti freghiamo: sai **in anticipo** come funziona la valutazione. Leggila una volta, poi lavora tranquillo.

1. Conta prima di tutto come ti poni. L'atteggiamento pesa quanto e più della tecnica: esserci in orario, non mollare al primo errore, dire “ho sbagliato” invece di nasconderti, dare una mano ai compagni. È la prima cosa che guardo — ed è la stessa che guarderà, un domani, chi ti darà un lavoro.

2. Il codice che funziona è il biglietto d'ingresso, non il voto. Consegnare il gioco che gira ti fa “entrare alla prova”. Ma il codice **da solo** non fa il voto: puoi copiarlo o fartelo scrivere... e allora non direbbe niente su di te.

3. Il voto nasce dalla prova dal vivo, da solo. Uno di questi due, o tutti e due:

- **Me lo spieghi:** racconti **a parole tue** cosa fa il tuo codice.
- **Il gioco rotto:** ti do il tuo gioco con **2-3 errori nascosti** dentro, e tu lo **rimetti in piedi lì per lì** — da solo, senza AI e senza chiedere ai compagni.

Se hai capito, li fai in pochi minuti. Se hai solo incollato, ti blocchi. Ecco perché **capire conviene**.

4. Il patto con l'AI e con i compagni. Puoi usarli per **imparare**: capire un errore, farti spiegare, avere uno spunto. Non per **consegnare senza capire**. La prova del nove è sempre la stessa: **lo sai spiegare e riparare da solo?**

5. Zero vergogna. Usare gli aiuti, come i 4 livelli, l'AI o un compagno, è **permesso e normale** — non è imbrogliare. Anche sbagliare è normale: **capita a tutti i programmatori**, pure ai più bravi. L'unica cosa che conta è che, alla fine, **tu abbia capito**.

Scrivere in Markdown

La tua dispensa, fatta con due segnetti.

Markdown, si dice “*marc-daun*”, è un modo per scrivere **testo normale** e aggiungere qualche **segnetto** che dice *come deve apparire*: questo è un titolo, questa parola è in grassetto, questo è un elenco.

***Il ponte con quello che conosci:** in Word premi il bottone grassetto; qui il bottone non c'è, il grassetto lo **scrivi tu** mettendo due asterischi attorno alla parola. Tutto qui. I segnetti li scrivi tu, ma nel risultato finale **non si vedono**.*

La tabella dei segnetti

Vuoi ottenere...	Scrivi così
Un titolo grande	# Il mio titolo
Un titolo più piccolo	## Sottotitolo — più #, più piccolo
Grassetto	**parola** — due asterischi prima e dopo
<i>Corsivo</i>	*parola* — un asterisco prima e dopo
Un punto elenco	- prima cosa — trattino più spazio
Un elenco numerato	1. prima cosa
Un nome tecnico / codice in riga	`_process` — un apice basso prima e dopo
Un'immagine, un tuo screenshot	![il mio gioco](immagini/es1.png)
Una nota/riquadro	> Attenzione: ricordati di salvare!

L'apice basso ```, in inglese *backtick*, su tastiera italiana si fa con **Alt Gr** + `'`, il tasto dell'apostrofo vicino allo `0`.

Un blocco di codice

Metti **tre apici bassi** prima e **tre** dopo: così il codice viene mostrato bello incolonnato.

```
```gdscript
func _ready():
 print("Ciao!")
```
```

Le 4 regole d'oro

1. **Riga vuota tra un paragrafo e l'altro.** Se non la metti, le frasi si attaccano tutte insieme.
2. **Uno spazio dopo # e dopo -.** `#Titolo` non funziona, `# Titolo` sì.
3. **I segnetti non si vedranno:** servono solo a dare la forma. Non spaventarti se nel testo grezzo sembrano strani.
4. **Bloccato? Fatti aiutare bene dall'AI.** Scrivi con **parole tue**, poi chiedi “*mettimi questo in Markdown*”, e **guarda come l'ha fatto** — così impari il segnetto per la prossima volta. Ricorda il patto: l'AI ti aiuta a *formattare*, il pensiero resta tuo.

Come parti da un modello

Non parti mai da un foglio bianco: c'è un **modello** già pronto, in inglese *template*, con i titoli e i posti da riempire.

Cos'è il “repository”? È solo una parola tecnica per dire **la cartella del corso su GitHub**, dove sono raccolti tutti i file: il manuale, gli esercizi, i modelli. È lo stesso posto che gestiamo con **Git**. Da browser lo apri come un sito qualsiasi: cartelle e file su cui puoi cliccare.

Ecco come apri il modello e te ne fai **una copia tua**:

1. `[BROWSER]` apri la pagina del corso su GitHub. L'indirizzo esatto te lo do io.
2. Nella lista, clicca prima la cartella `manuale`, poi il file `quaderno-studente-TEMPLATE.md`: si apre e vedi il testo del modello.
3. In alto a destra, sopra il testo, clicca l'iconcina `Copy raw file`, che copia tutto il contenuto in un colpo solo.
4. Torna nel **TUO** repository, la tua copia personale → bottone `Add file` → `Create new file`.
5. Dai un nome che finisce con `.md`, per esempio `es1.md`.
6. **Incolla** con `Ctrl + V` il modello, poi **riempi i vuoti** con le tue cose.
7. Bottone verde `Commit changes` per salvare. Fatto!

Come metto un mio screenshot

Uno screenshot è una **foto dello schermo**. Attenzione: appena lo fai, finisce solo nella memoria temporanea, i cosiddetti “appunti” — **non è ancora un file** sul computer. Ecco tutti i passaggi:

1. [Windows] premi **Win + Shift + S**: lo schermo si scurisce, **trascina** un riquadro attorno al tuo gioco. L'immagine viene copiata.
2. In basso a destra compare un **avviso: cliccaci sopra** → si apre lo **Strumento di cattura**.
3. Lì clicca l'iconcina del **dischetto** per salvare, scegli una cartella che **ricordi bene**, per esempio il **Desktop**, e un nome che finisce con **.png**, per esempio **es1.png**. Ora è un **file** sul tuo computer.
4. [BROWSER] nel tuo repository apri la cartella **immagini/** → bottone **Add file** → **Upload files** → **trascina** dentro il tuo **es1.png**.
5. La riga nel modello è **già pronta**: **![il mio gioco](immagini/es1.png)** → l'immagine comparirà da sola.

Prova del nove: se guardi la tua pagina e vedi il titolo grande, il grassetto e il tuo screenshot al posto giusto... **ce l'hai fatta!**

La tua copia del corso, e come consegnare

Il fork: il tuo spazio, dove lavori in sicurezza.

Per lavorare **non tocchi** il corso del prof: te ne fai una **copia tutta tua**, dove nessuno può rovinare il tuo lavoro e tu non rovini quello degli altri. In GitHub questa copia si chiama **fork**.

Passo 1 — Fatti la tua copia, il fork

1. `[BROWSER]` entra su GitHub con il **tuo account** e apri la pagina del corso (l'indirizzo te lo do io).
2. In alto a destra clicca il bottone `Fork`, poi `Create fork`.
3. Fatto: in alto ora c'è il **tuo nome** — quella è la **tua copia** del corso.

Passo 2 — Prepara la consegna

Ogni consegna è una **cartella** dentro `consegne/`, con tre cose: la **scheda** compilata, il tuo **codice** e i tuoi **screenshot**.

1. Nella tua copia apri `consegne/_MODELLO/scheda.md` e clicca `Copy raw file`.
2. Bottone `Add file` → `Create new file`. Come nome scrivi il percorso con la cartella dell'esercizio, per esempio `consegne/es1-bottone/scheda.md`. **Incolla** la scheda e **riempila**. In fondo, `Commit`.
3. Bottone `Add file` → `Upload files`: trascina il tuo `main.gd` e gli screenshot, poi `Commit`.

Passo 3 — Consegna

Manda al prof il **link** della tua cartella. Lui la corregge, mette il voto e la aggiunge al **tuo** libro personale.

*Se ti blocchi, dai a Gemini il file `ISTRUZIONI.md` che trovi nel kit: sa guidarti passo passo. Ma la parte “cosa ho fatto con parole mie” scrivila **da solo** — è quella che conta.*

Cos'è Godot, il parente di Lazarus

Godot è un programma gratuito per creare **giochi** e app interattive. È molto simile, come spirito, a **Lazarus**: entrambi sono ambienti gratuiti dove **componi qualcosa a schermo e ci attacchi del codice**.

La “stele di Rosetta” Lazarus → Godot:

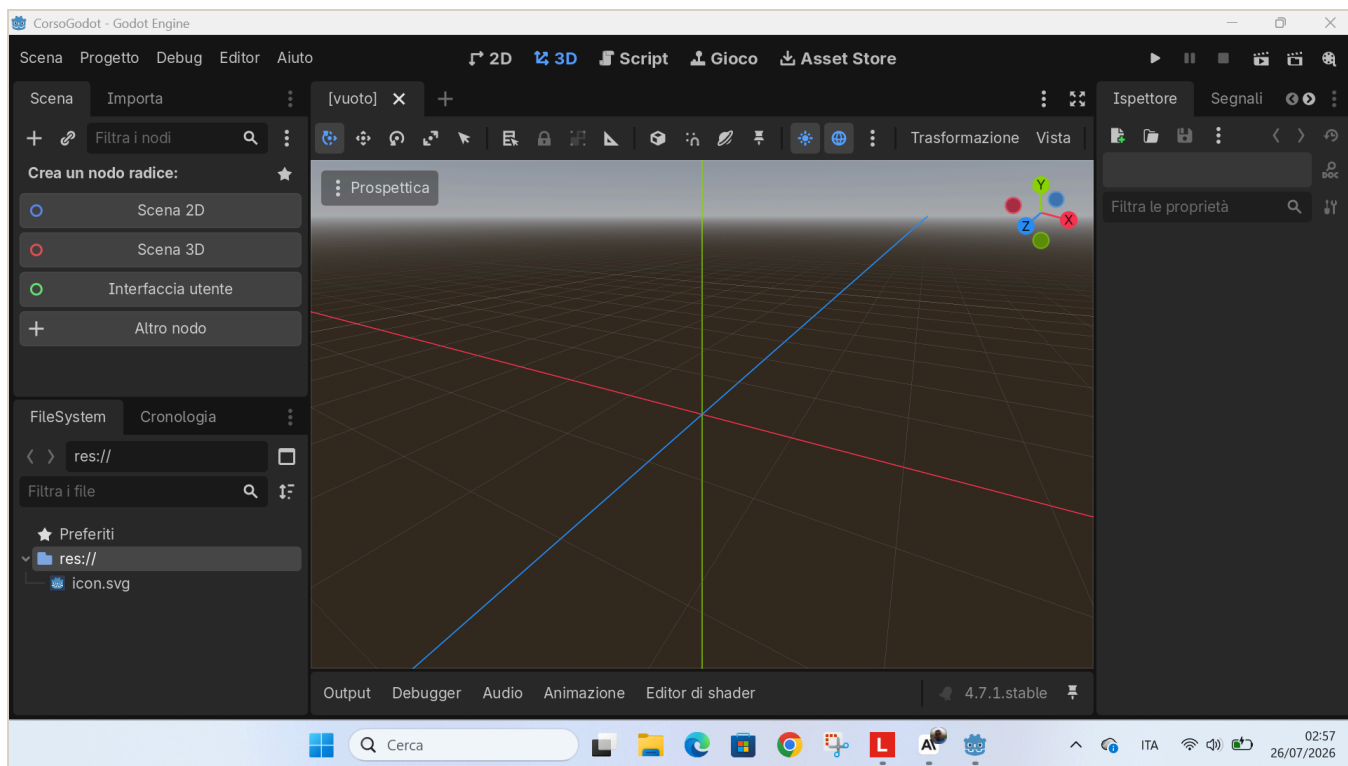
| In Lazarus, che già conosci | In Godot, la novità |
|--|---|
| Progetto | Progetto , identico |
| Form , la finestra | Scena , un <i>albero di nodi</i> più potente |
| Componenti : TButton, TEdit... | Nodi : Button, LineEdit, Label... |
| Proprietà come Caption, nell'Object Inspector | Proprietà nell' Ispettore |
| Gestore evento , <code>Button1Click</code> | Segnale + funzione |
| Object Pascal , il linguaggio | GDScript , in stile Python |

La **differenza più importante — il “game loop”**: in Lazarus il programma è **fermo** finché non clicchi qualcosa. In Godot c'è una funzione, `_process(delta)`, che **gira da sola ~60 volte al secondo**, in continuazione. È questo che fa muovere le cose: personaggi, oggetti che cadono, animazioni.

*In una frase: **Lazarus reagisce, Godot pulsa.***

L'ambiente di sviluppo all'avvio

Quando apri un progetto nuovo, l'editor si presenta così: vista **3D** di default e scena ancora vuota.



L'editor di Godot appena aperto, con la scena vuota

Per il nostro corso lavoreremo quasi sempre in **2D**: cliccheremo “**Scena 2D**” per iniziare. Lo vedrai nei capitoli in cui costruiamo gli esercizi.

I 4 concetti base di Godot

Se capisci questi quattro, capisci Godot:

| Concetto | Cos'è | Analogia LEGO |
|--|---|--------------------|
| Progetto | La cartella con dentro il file <code>project.godot</code> e tutto il gioco | La scatola del set |
| Nodo | Il mattoncino base: Sprite2D per un'immagine, Label per un testo, Timer per un cronometro | Un pezzo di LEGO |
| Scena | Tanti nodi messi ad albero, salvati in un file <code>.tscn</code> | Una costruzione |
| Script , il file <code>.gd</code> | Codice GDScript attaccato a un nodo, che gli dà comportamento | Le istruzioni |

Regola d'oro: in Godot **tutto è un nodo**; le scene sono nodi messi insieme; gli script danno vita ai nodi.

GDScript: il linguaggio

GDScript è la lingua che si parla **dentro** Godot. È stato fatto apposta per somigliare a **Python**: si legge facile, si usa il **rientro con TAB** per raggruppare le righe, **niente punto e virgola**.

Due funzioni speciali che Godot chiama da solo:

- `func _ready():` → eseguita **una volta**, all'avvio, per preparare le cose.
- `func _process(delta):` → eseguita **a ogni fotogramma**: è il game loop. `delta` = secondi passati dall'ultimo fotogramma; serve a muoversi in modo fluido su qualsiasi PC.

Esempio minimo, leggere le frecce e spostare qualcosa:

```
func _process(delta):  
    if Input.is_action_pressed("ui_left"):  
        posizione.x -= 200 * delta    # vai a sinistra  
    if Input.is_action_pressed("ui_right"):  
        posizione.x += 200 * delta    # vai a destra
```

I mattoncini, uno alla volta

Prima di costruire un gioco intero, impariamo i pezzi **uno per uno**. Ogni passo funziona così: **impari una cosa sola**, la **provi subito** a schermo, e vai avanti. Regola: non passare al passo dopo finché quello di prima non funziona. Aprilo così, una volta per tutte: [APP – Godot] finestra iniziale, il *Gestore progetti*, in alto a destra **Crea**, dai un nome tipo **prove**, scegli una cartella e **Crea e modifica**. Da qui in poi lavoriamo dentro questo progetto.

Passo 1 — La scena e il primo nodo

Cosa impari: una **scena** è un albero di **nodi**, i mattoncini. Mettiamone uno.

Provalo tu: [APP – Godot] → pannello **Scena** in alto a sinistra → premi il **+** (Aggiungi nodo figlio) → scrivi e scegli **Label** → nel pannello **Ispettore** a destra, alla proprietà **Text**, scrivi **Ciao**. Premi **F5** (la prima volta ti chiede di salvare la scena: dai **Salva**).

Fatto: se sullo schermo compare **Ciao**, hai messo il tuo primo nodo.

Passo 2 — Una proprietà

Cosa impari: ogni nodo ha delle **proprietà** e le cambi nell’Ispettore. È come la **Caption** di Lazarus, ma ce ne sono tante.

Provalo tu: seleziona il **Label** nel pannello Scena → nell’**Ispettore** cerca **Theme Overrides** → **Font Sizes** → **Font Size** e mettilo a **48**. Cambia anche il **Text** con il tuo nome. Premi **F5**.

Fatto: hai cambiato una proprietà e l’hai vista cambiare.

Passo 3 — Lo script e `_ready()`

Cosa impari: uno **script** è un file **.gd** attaccato a un nodo, che gli dà comportamento. La funzione `_ready()` gira **una volta**, all’avvio.

Provalo tu: seleziona il **nodo radice** (in cima all’albero) → nel pannello Scena, in alto, premi l’icona **Aggiungi script** → **Crea**. Nel codice, dentro `func _ready():`, scrivi una riga:

```
print("Parto!")
```

Premi **F5**: guarda in basso il pannello **Output**, deve comparire *Parto!*.

Fatto: hai attaccato uno script e visto partire `_ready()`.

Passo 4 — Una variabile

Cosa impari: una **variabile** è una scatola con un nome, che tiene un valore.

Provalo tu: in **cima** allo script scrivi una riga, poi stampala:

```
var punti = 0

func _ready():
    print(punti)
```

Premi **F5**: nell'Output compare `0`.

Fatto: hai creato una variabile e mostrato quello che contiene.

Passo 5 — Dare un soprannome a un nodo con `@onready`

Cosa impari: per comandare un nodo dal codice ti serve un **soprannome** per prenderlo. Si scrive in cima allo script:

```
@onready var etichettaVar: Label = $etichettaScena
```

Da sinistra: `etichettaVar` è il **soprannome** che usi nel codice; `$etichettaScena` è il **nodo vero** nella scena; `@onready` vuol dire “prendilo appena la scena è pronta”.

Provalo tu: nel pannello Scena, doppio clic sul **Label** e rinominalo `etichettaScena`. Poi metti la riga qui sopra in cima allo script, e in `_ready()`:

```
etichettaVar.text = "Funziona!"
```

Premi **F5**: se compare *Funziona!*, hai finito.

Fatto: sai prendere un nodo e comandarlo dal codice.

Passo 6 — Un segnale e la sua funzione

Cosa impari: un **segnale** è un annuncio del nodo (“mi hanno premuto”). Lo **colleghi** a una tua **funzione**. È il vecchio `Button1Click` di Lazarus.

Provalo tu: aggiungi un nodo **Button** (pannello Scena → + → **Button**), rinominalo **bottoneScena**, dagli **Text** = **Premimi**. Nello script:

```
@onready var bottoneVar: Button = $bottoneScena

func _ready():
    bottoneVar.pressed.connect(_quando_premo)

func _quando_premo():
    etichettaVar.text = "Premuto!"
```

Premi **F5** e clicca il bottone.

Fatto: hai collegato un **evento** a una funzione tua.

Passo 7 — Il game loop **_process(delta)**

Cosa impari: **_process(delta)** gira **da solo**, circa 60 volte al secondo. È la grande novità rispetto a Lazarus: *Lazarus reagisce, Godot pulsa*. **delta** è il tempo passato dall'ultimo fotogramma: serve a muoversi fluidi su qualsiasi PC.

Provalo tu: aggiungi questa funzione allo script:

```
func _process(delta):
    etichettaVar.position.x += 100 * delta
```

Premi **F5**: la scritta **scivola da sola** verso destra.

Fatto: hai visto il game loop muovere qualcosa senza che tu prema niente.

Passo 8 — Leggere i tasti con **Input**

Cosa impari: **Input.is_action_pressed("ui_right")** è vero **mentre** tieni premuta la freccia destra. Così comandi tu.

Provalo tu: cambia il **_process** così:

```
func _process(delta):
    if Input.is_action_pressed("ui_right"):
        etichettaVar.position.x += 200 * delta
    if Input.is_action_pressed("ui_left"):
        etichettaVar.position.x -= 200 * delta
```

Premi **F5** e tieni premute le frecce ← →.

Fatto: muovi un nodo con la tastiera.

Passo 9 — Decidere con `if`

Cosa impari: `if` fa una cosa **solo se** una condizione è vera. Serve a mettere delle regole.

Provalo tu: in fondo al `_process`, aggiungi una regola che ferma la scritta al bordo:

```
if etichettaVar.position.x > 400:
    etichettaVar.position.x = 400
```

Premi `F5`: la scritta si muove ma **non supera** il bordo.

Fatto: il tuo programma prende una **decisione** da solo.

Passo 10 — Un interruttore acceso o spento: il booleano

Cosa impari: un **booleano** è una variabile che vale solo `true` (vero) o `false` (falso), come un interruttore. Serve a ricordare uno **stato**, per esempio se il gioco è in corso oppure è finito.

Provalo tu: in cima allo script metti una variabile interruttore, e usala nel `_process` per far muovere la scritta e poi fermarla da sola:

```
var in_gioco: bool = true

func _process(delta):
    if in_gioco:
        etichettaVar.position.x += 100 * delta
    if etichettaVar.position.x > 400:
        in_gioco = false
```

Premi `F5`: la scritta scivola e poi **si ferma** quando `in_gioco` diventa `false`.

Fatto: hai usato un interruttore per accendere e spegnere un comportamento. È così che un gioco sa se **sta giocando** o è **finito**.

Passo 11 — Far comparire e sparire un nodo con `visible`

Cosa impari: ogni nodo ha la proprietà `visible`. Se è `false` il nodo **sparisce**, se è `true` **ricompare**. È il trucco per la scritta GAME OVER: sta nascosta e appare solo alla fine.

Provalo tu: nascondi la scritta all'avvio.


```
func _ready():  
    etichettaVar.text = "FINE"  
    etichettaVar.visible = false
```

Premi **F5**: la scritta **non si vede**. Se più avanti metti `etichettaVar.visible = true`, ricompare.

Fatto: sai nascondere e mostrare un nodo a comando.

Ora hai tutti i mattoncini in mano: **nodo**, **proprietà**, **script**, `_ready`, **variabile**, **soprannome** `@onready`, **segnale**, **game loop**, `Input`, `if`, **booleano**, `visible`. Con questi, il prossimo capitolo, dove costruiamo l'Esercizio 1, non è più un salto nel buio: sono le stesse cose, messe insieme.

Costruiamo l'Esercizio 1: il bottone che saluta

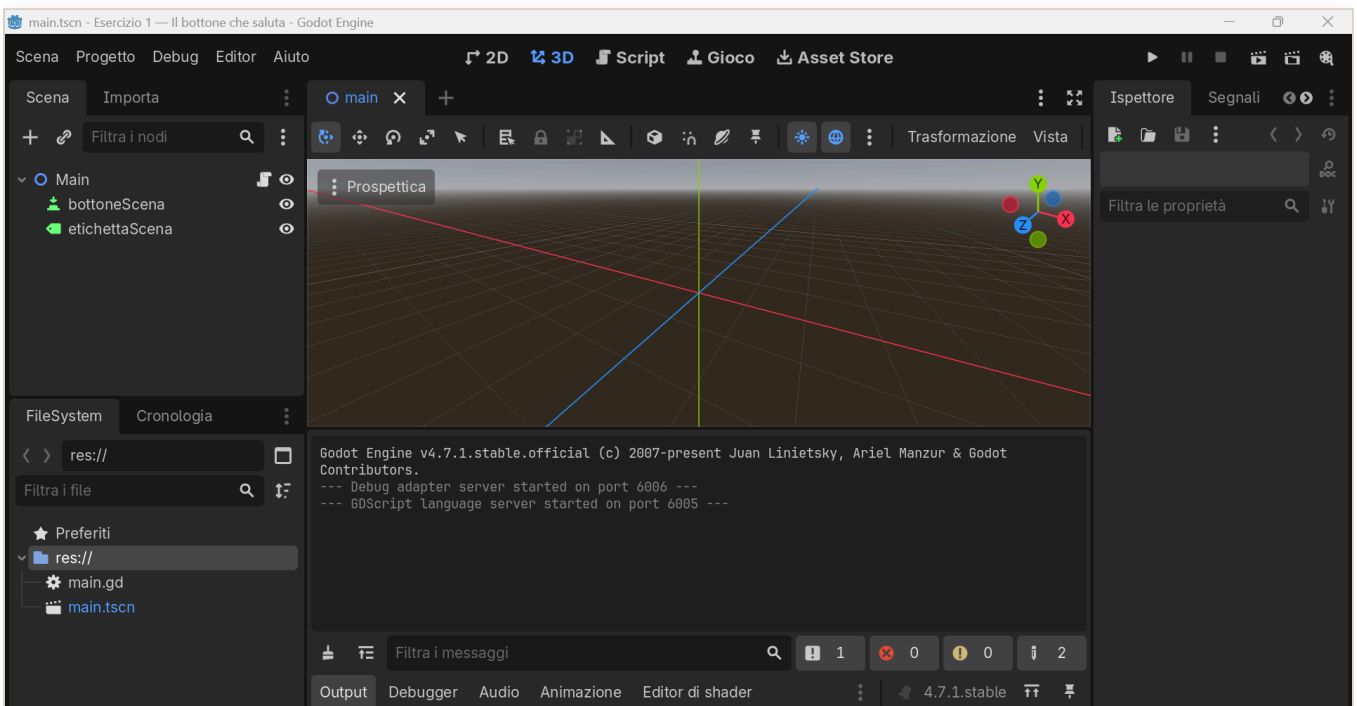
Il tuo primo gioco vero, in pochi passi. Alla fine avrai **un bottone** che, quando lo premi, **ti saluta**. È il ponte da Lazarus: il vecchio `Button1Click` qui diventa un **segnale**.

Passo 1 — Crea il progetto

[APP – Godot] nella finestra iniziale, il Gestore progetti, in alto a destra clicca **Crea**. Dai un nome, per esempio `esercizio01`, scegli una cartella e premi **Crea e modifica**.

Passo 2 — Fai la scena

[APP – Godot] pannello **Scena**, in alto a sinistra, clicca **Scena 2D**: nasce il nodo radice `Node2D`. Con un **doppio clic** sul suo nome, rinominalo **Main**.



L'editor di Godot con la scena: i nodi Main, bottoneScena ed etichettaScena

Passo 3 — Aggiungi il bottone e la scritta

1. Seleziona **Main**, poi in alto nel pannello Scena clicca il **+**, cioè Aggiungi nodo figlio. Cerca **Button**, selezionalo, **Crea**. Rinominalo **bottoneScena**.
2. Seleziona di nuovo **Main**, **+**, cerca **Label**, **Crea**. Rinominalo **etichettaScena**.

Passo 4 — Attacca lo script

Seleziona **Main**. In alto a destra del pannello Scena clicca **Attacca uno script**, l'icona con la pergamena e il **+**. Lascia tutto com'è e premi **Crea**.

Passo 5 — Scrivi il codice

Cancella quello che trovi e incolla:

```
extends Node2D

@onready var bottoneVar: Button = $bottoneScena
@onready var etichettaVar: Label = $etichettaScena

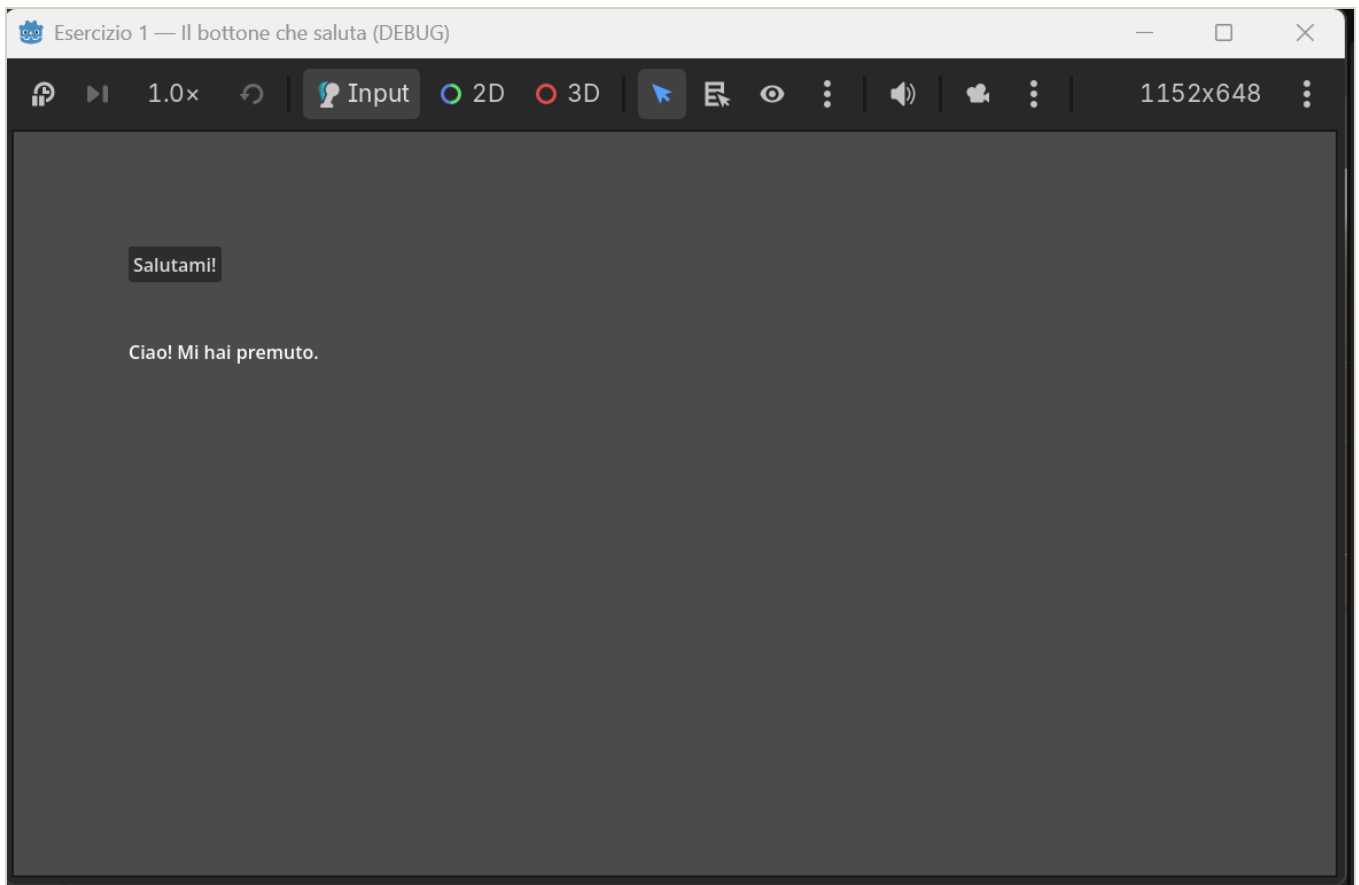
func _ready() -> void:
    bottoneVar.position = Vector2(100, 100)
    bottoneVar.text = "Salutami!"
    etichettaVar.position = Vector2(100, 180)
    etichettaVar.text = "..."
    # Colleghiamo il clic, il segnale pressed, alla nostra funzione
    bottoneVar.pressed.connect(_quando_premo)

# Questa è come il tuo Button1Click di Lazarus
func _quando_premo() -> void:
    etichettaVar.text = "Ciao! Mi hai premuto."
```

Pezzo per pezzo: **@onready var** prende i due nodi per nome; in **_ready()** diamo posizione e testo; **bottone.pressed.connect(...)** collega il **clic** alla funzione **_quando_premo**, che cambia la scritta.

Passo 6 — Vinci

Premi **F5**. Si apre la finestra: **premi il bottone** e compare “Ciao! Mi hai premuto.” **Ce l’hai fatta.**



Il gioco che saluta

Fallo tuo

- Cambia la frase del saluto con **la tua**.
- Dai un colore alla scritta: dentro `_ready()` aggiungi questa riga, dove i tre numeri sono rosso, verde, blu da 0 a 1: `gdscript etichettaVar.add_theme_color_override("font_color", Color(1, 0, 0))`

Costruiamo l'Esercizio 2: muovi il quadrato

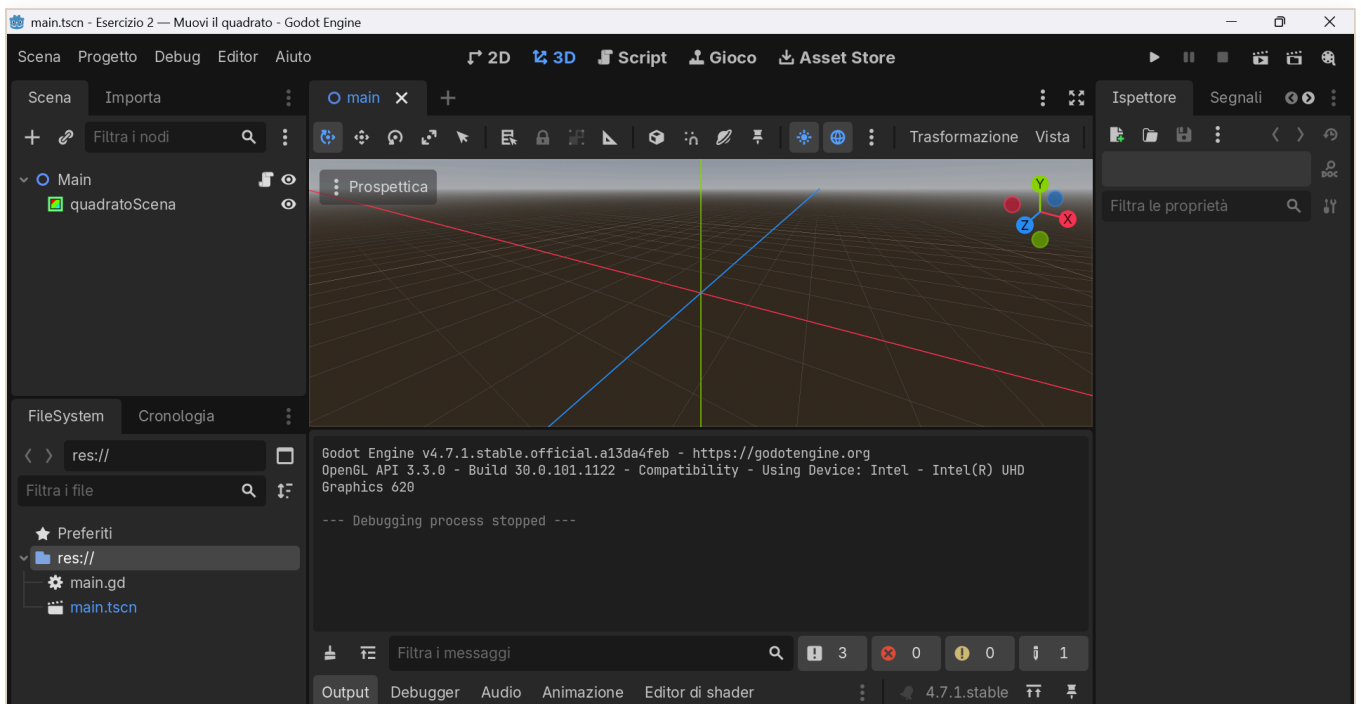
Qui incontri il concetto più importante di Godot, il **game loop**. Alla fine avrai un **quadrato** che si muove con le frecce.

Passo 1 — Progetto e scena

Come prima: crea un progetto `esercizio2`, poi nel pannello Scena clicca **Scena 2D** e rinomina il nodo radice **Main**.

Passo 2 — Aggiungi il quadrato

Seleziona **Main**, clicca **+**, cerca **ColorRect**, cioè un rettangolo colorato, **Crea**, e rinominalo **quadratoScena**.



L'editor di Godot con la scena: i nodi Main e quadratoScena

Passo 3 — Script e codice

Attacca lo script a **Main**, cancella e incolla:

```

extends Node2D

@onready var quadratoVar: ColorRect = $quadratoScena
const VELOCITA: float = 300.0 # pixel al secondo

func _ready() -> void:
    quadratoVar.size = Vector2(60, 60)
    quadratoVar.color = Color(0.3, 0.7, 1.0) # azzurro
    quadratoVar.position = Vector2(200, 200)

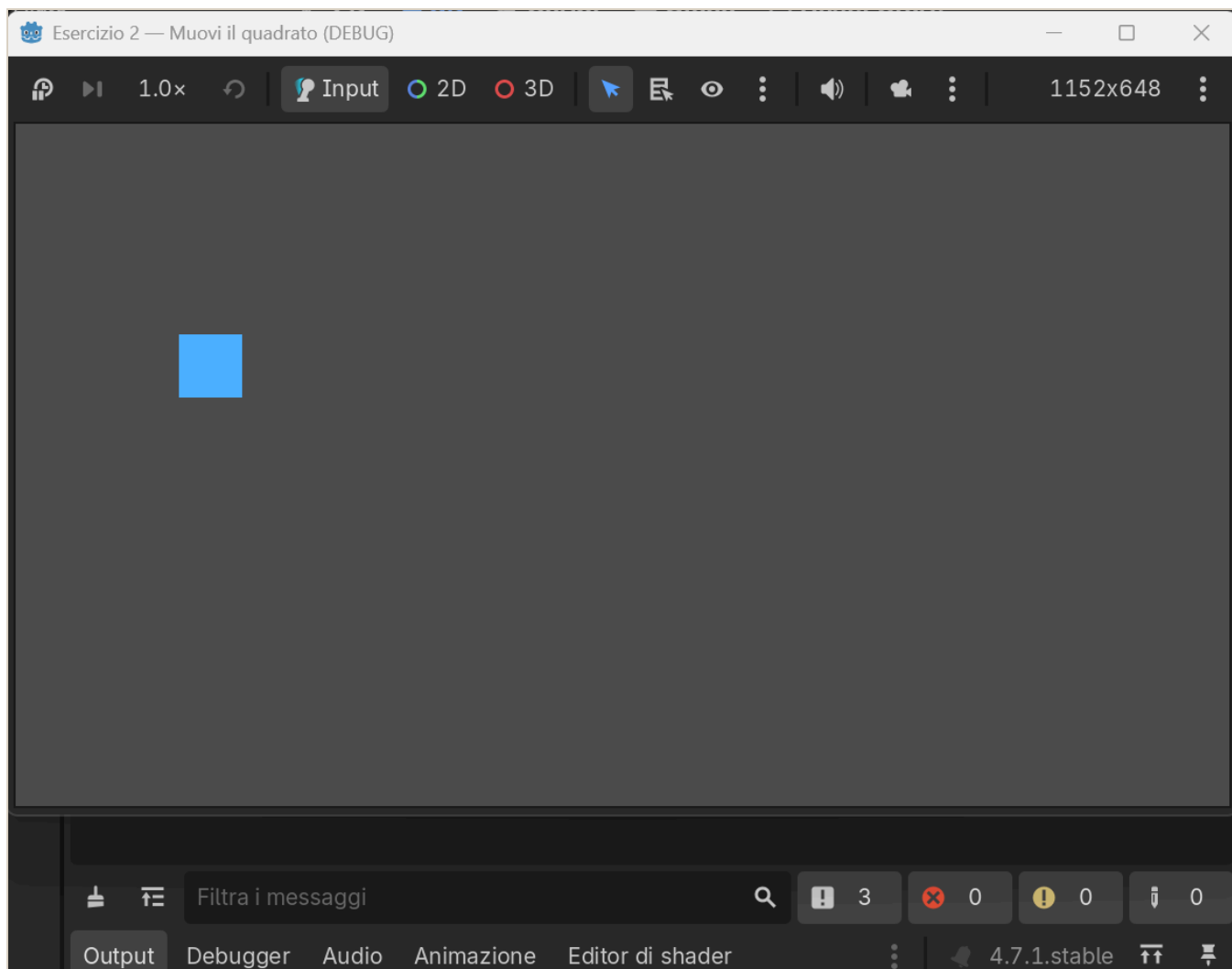
# _process gira a OGNI fotogramma: qui muoviamo il quadrato
func _process(delta: float) -> void:
    if Input.is_action_pressed("ui_left"):
        quadratoVar.position.x -= VELOCITA * delta
    if Input.is_action_pressed("ui_right"):
        quadratoVar.position.x += VELOCITA * delta
    if Input.is_action_pressed("ui_up"):
        quadratoVar.position.y -= VELOCITA * delta
    if Input.is_action_pressed("ui_down"):
        quadratoVar.position.y += VELOCITA * delta

```

La novità è `func _process(delta):`, che **gira da sola ~60 volte al secondo**. Dentro controlliamo le frecce con `Input.is_action_pressed(...)` e spostiamo il quadrato. `delta` serve a muoversi uguale su ogni computer.

Passo 4 — Vinci

F5, e muovi il quadrato con le **frecce**. *Lazarus reagisce, Godot pulsa.*



Il quadrato che si muove

Fallo tuo

- Cambia il **colore** in `quadrato.color = Color(...)`.
- Cambia la **velocità**: il numero in `const VELOCITA`.

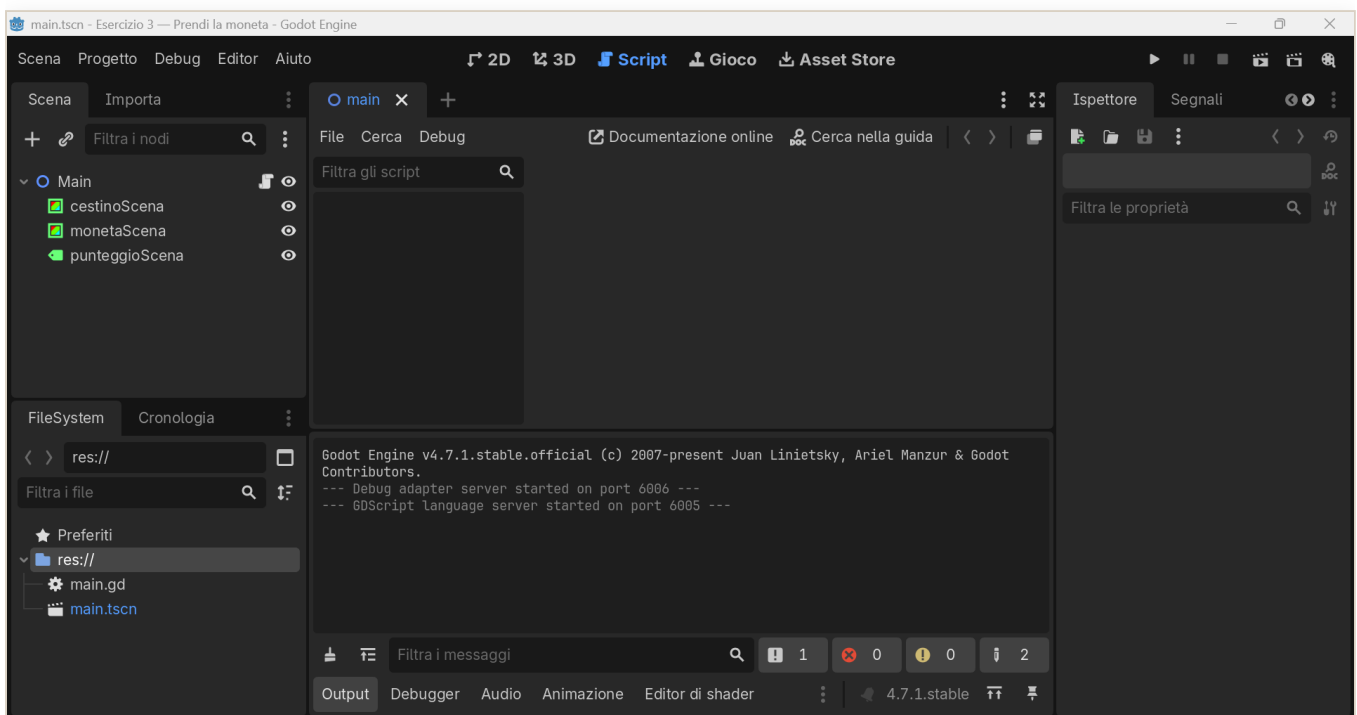
Costruiamo l'Esercizio 3: prendi la moneta

Il primo **mini-gioco vero**: un **cestino** che prende le **monete** che cadono, con **punteggio**. Mette insieme movimento, collisioni e punteggio. È l'idea del vecchio “Chirurgo Pasticcione”, ma più pulita.

Passo 1 — Scena

Crea il progetto `esercizio3`, **Scena 2D**, nodo radice **Main**. Poi aggiungi tre figli a **Main** con il **+**:

- **ColorRect** → rinominalo **cestinoScena**
- **ColorRect** → rinominalo **monetaScena**
- **Label** → rinominalo **punteggioScena**



L'editor di Godot con la scena: i nodi cestinoScena, monetaScena e punteggioScena

Passo 2 — Script e codice

Attacca lo script a **Main**, cancella e incolla:


```
extends Node2D
```

```
@onready var cestinoVar: ColorRect = $cestinoScena
```

```
@onready var monetaVar: ColorRect = $monetaScena
```

```
@onready var punteggioVar: Label = $punteggioScena
```

```
const VELOCITA_CESTINO: float = 500.0
```

```
const VELOCITA_MONETA: float = 300.0
```

```
var punti: int = 0
```

```
var larghezza: float
```

```
func _ready() -> void:
```

```
    larghezza = get_viewport_rect().size.x
```

```
    cestinoVar.size = Vector2(120, 24)
```

```
    cestinoVar.color = Color(0.6, 0.4, 0.2)
```

```
    cestinoVar.position = Vector2(larghezza / 2.0 - 60, get_viewport_rect().size.y - 60)
```

```
    monetaVar.size = Vector2(30, 30)
```

```
    monetaVar.color = Color(1.0, 0.85, 0.1)
```

```
    _rimetti_in_alto()
```

```
    punteggioVar.position = Vector2(20, 20)
```

```
    _aggiorna_punteggio()
```

```
func _process(delta: float) -> void:
```

```
    if Input.is_action_pressed("ui_left"):
```

```
        cestinoVar.position.x -= VELOCITA_CESTINO * delta
```

```
    if Input.is_action_pressed("ui_right"):
```

```
        cestinoVar.position.x += VELOCITA_CESTINO * delta
```

```
    cestinoVar.position.x = clamp(cestinoVar.position.x, 0, larghezza -  
cestinoVar.size.x)
```

```
    monetaVar.position.y += VELOCITA_MONETA * delta
```

```
    # Il cestino tocca la moneta?
```

```
    if Rect2(cestinoVar.position, cestinoVar.size).intersects(Rect2(monetaVar.position,  
monetaVar.size)):
```

```
        punti += 1
```

```
        _aggiorna_punteggio()
```

```
        _rimetti_in_alto()
```

```
    elif monetaVar.position.y > get_viewport_rect().size.y:
```

```
        _rimetti_in_alto()
```

```
func _rimetti_in_alto() -> void:
```

```
    var x := randf_range(0, larghezza - monetaVar.size.x)
```

```
    monetaVar.position = Vector2(x, -monetaVar.size.y)
```

```
func _aggiorna_punteggio() -> void:
```

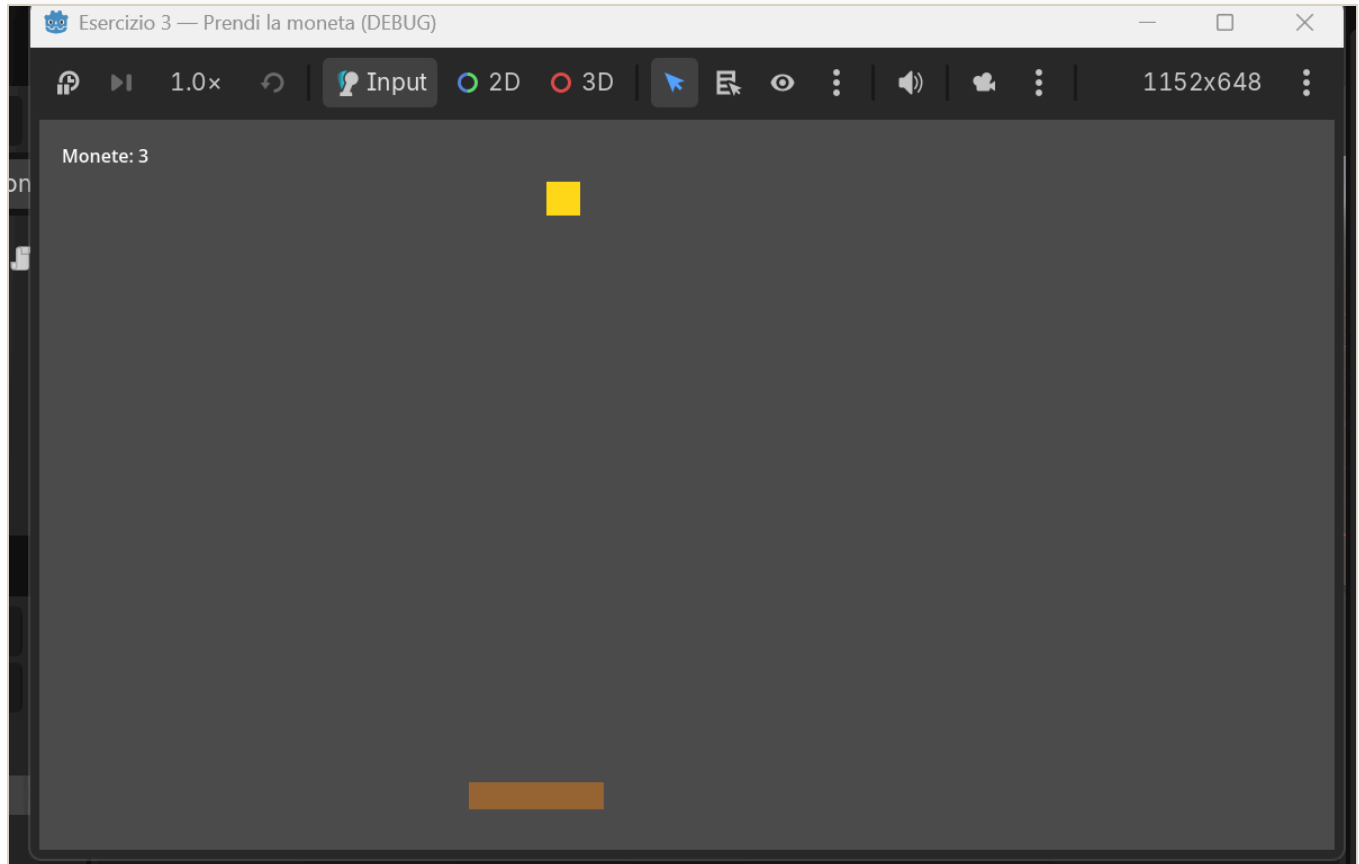
```
    punteggioVar.text = "Monete: %d" % punti
```

Cosa succede: in `_ready()` prepariamo cestino, moneta e punteggio; in `_process()` muoviamo il cestino con le frecce, facciamo scendere la moneta e con `Rect2(...).intersects(...)` controlliamo

se il cestino **tocca** la moneta. Se sì, **+1 punto** e la moneta riparte dall'alto in una colonna a caso.

Passo 3 — Vinci

F5: muovi il cestino con **←** **→** e **prendi le monete**. Il punteggio sale.



Il gioco della moneta

Fallo tuo

- Cambia i **colori** di cestino e moneta.
- Rendilo più difficile: aumenta `VELOCITA_MONETA`.
- Aggiungi le **vite** e un “Game Over” quando finiscono, come nel vecchio “Chirurgo Pasticcione”. Lo costruiamo insieme nel prossimo capitolo.

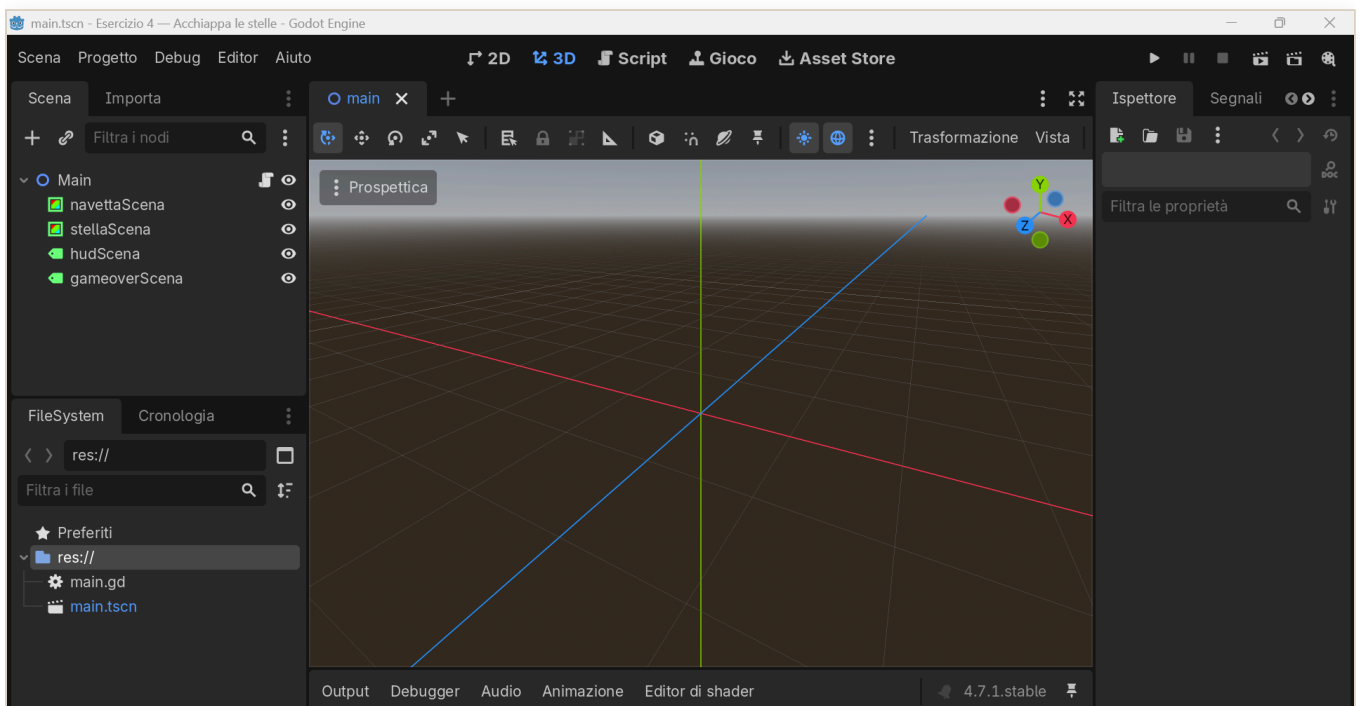
Costruiamo l'Esercizio 4: acchiappa le stelle

È l'Esercizio 3 che **cresce**. Stessa idea, una navetta che prende le stelle che cadono, ma ora il gioco si può **perdere**: hai tre **vite**, e se lasci cadere troppe stelle è **game over**. Poi premi INVIO e ricominci. I concetti nuovi li hai già provati nel Capitolo 3: il **booleano** per lo stato e `visible` per far comparire la scritta di fine partita.

Passo 1 — Scena

Crea il progetto `esercizio4`, **Scena 2D**, nodo radice **Main**. Aggiungi a **Main** quattro figli con il **+**:

- **ColorRect** → rinominalo **navettaScena**
- **ColorRect** → rinominalo **stellaScena**
- **Label** → rinominalo **hudScena**
- **Label** → rinominalo **gameoverScena**



L'editor di Godot con la scena: i nodi navettaScena, stellaScena, hudScena e gameoverScena

Passo 2 — Script e codice

Attacca lo script a **Main**, cancella tutto e incolla:

```
extends Node2D

@onready var navettaVar: ColorRect = $navettaScena
@onready var stellaVar: ColorRect = $stellaScena
@onready var hudVar: Label = $hudScena
@onready var gameoverVar: Label = $gameoverScena

const VELOCITA_NAVETTA: float = 500.0
const VELOCITA_STELLA: float = 300.0
const VITE_INIZIALI: int = 3

var punti: int = 0
var vite: int = VITE_INIZIALI
var in_gioco: bool = true
var larghezza: float

func _ready() -> void:
    larghezza = get_viewport_rect().size.x
    navettaVar.size = Vector2(90, 20)
    navettaVar.color = Color(0.3, 0.7, 1.0)
    navettaVar.position = Vector2(larghezza / 2.0 - 45, get_viewport_rect().size.y - 50)
    stellaVar.size = Vector2(28, 28)
    stellaVar.color = Color(1.0, 0.85, 0.2)
    hudVar.position = Vector2(20, 20)
    gameoverVar.position = Vector2(larghezza / 2.0 - 140, get_viewport_rect().size.y /
2.0 - 20)
    gameoverVar.text = "GAME OVER\nPremi INVIO per ricominciare"
    gameoverVar.visible = false
    _rimetti_in_alto()
    _aggiorna_hud()

func _process(delta: float) -> void:
    # Se la partita è finita: aspetta INVIO per ricominciare, e basta.
    if not in_gioco:
        if Input.is_action_just_pressed("ui_accept"):
            _ricomincia()
            return
        if Input.is_action_pressed("ui_left"):
            navettaVar.position.x -= VELOCITA_NAVETTA * delta
        if Input.is_action_pressed("ui_right"):
            navettaVar.position.x += VELOCITA_NAVETTA * delta
        navettaVar.position.x = clamp(navettaVar.position.x, 0, larghezza -
navettaVar.size.x)
        stellaVar.position.y += VELOCITA_STELLA * delta
    # Presa?
    if Rect2(navettaVar.position, navettaVar.size).intersects(Rect2(stellaVar.position,
stellaVar.size)):
```

```

    punti += 1
    _aggiorna_hud()
    _rimetti_in_alto()
# Persa: toglì una vita
elif stellaVar.position.y > get_viewport_rect().size.y:
    vite -= 1
    _aggiorna_hud()
    _rimetti_in_alto()
    if vite <= 0:
        _game_over()

func _rimetti_in_alto() -> void:
    var x := randf_range(0, larghezza - stellaVar.size.x)
    stellaVar.position = Vector2(x, -stellaVar.size.y)

func _game_over() -> void:
    in_gioco = false
    gameoverVar.visible = true

func _ricomincia() -> void:
    punti = 0
    vite = VITE_INIZIALI
    in_gioco = true
    gameoverVar.visible = false
    _rimetti_in_alto()
    _aggiorna_hud()

func _aggiorna_hud() -> void:
    hudVar.text = "Punti: %d    Vite: %d" % [punti, vite]

```

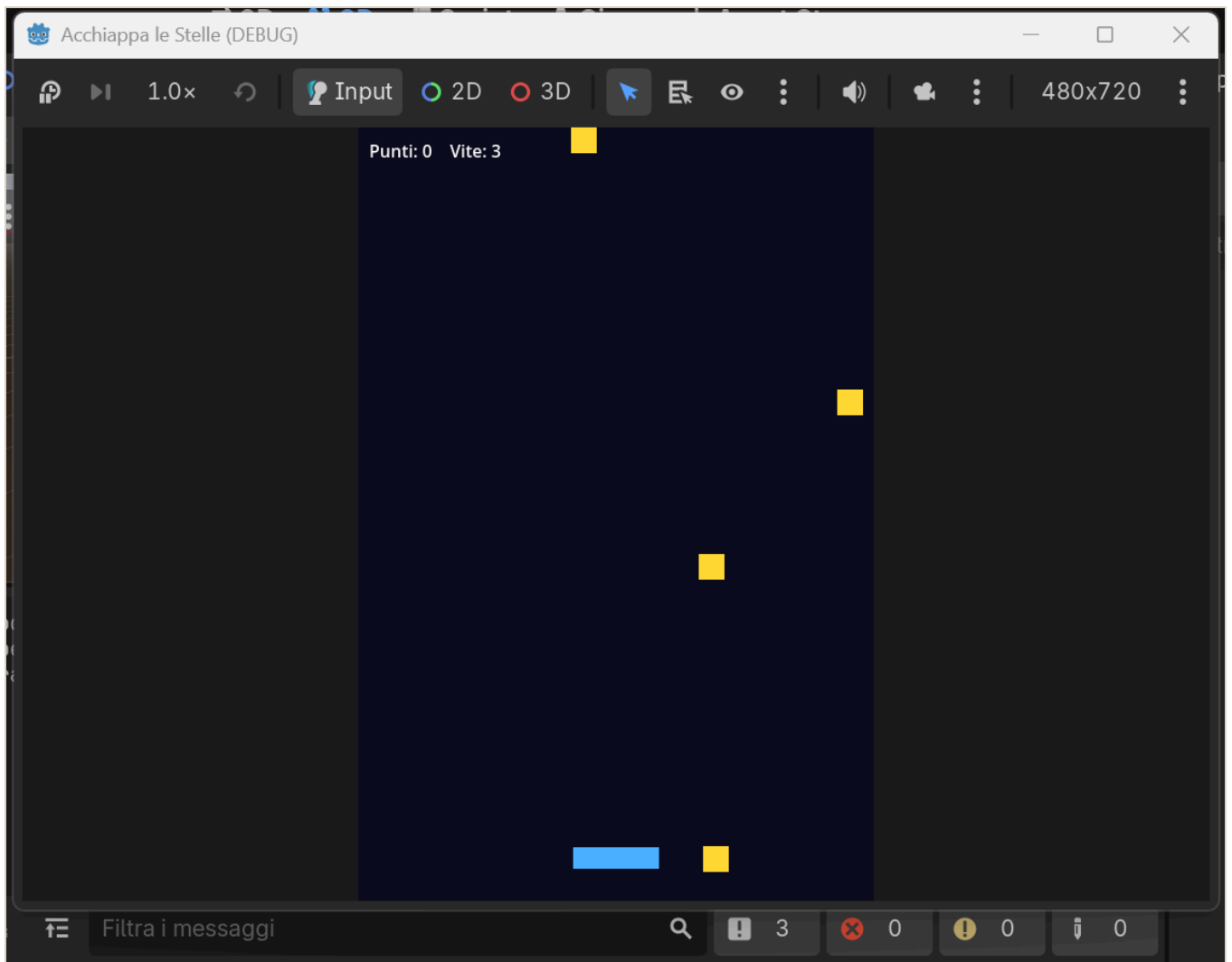
Cosa succede: teniamo lo **stato** del gioco in un interruttore, `in_gioco`. Finché è `true` giochi: muovi la navetta, la stella scende, se la prendi fai punto, se la perdi togli una **vita**. Quando le vite arrivano a zero chiamiamo `_game_over()`, che spegne `in_gioco` e **fa comparire** la scritta con `visible = true`. Da fermo, `Input.is_action_just_pressed("ui_accept")` aspetta un colpo di **INVIO** per ricominciare.

Due parole nuove da notare:

- `is_action_just_pressed` scatta **una volta sola** nell'istante in cui premi, non di continuo come `is_action_pressed`. Perfetto per “premi INVIO per ripartire”.
- `visible` è l'interruttore che fa **comparire o sparire** un nodo.

Passo 3 — Vinci

F5: muovi la navetta con `← →`, prendi le stelle, tieni le vite. Quando finiscono arriva il **GAME OVER**; premi **INVIO** e riparti.



Acchiappa le stelle: la navetta prende le stelle mentre scendono, con Punti e Vite in alto

Fallo tuo

- Cambia quante **vite** hai: il numero in `const VITE_INIZIALI`.
- Cambia i **colori** di navetta e stella, e la scritta di fine partita.
- Rendilo più difficile a poco a poco: aumenta un pochino `VELOCITA_STELLA` ogni volta che fai punto.

Il percorso: dagli esercizi al “progetto boss”

Qui non si impara con la teoria astratta, ma **facendo**. Ogni esercizio insegna **un pezzo**; poi arriva un gioco più grande — il “**progetto boss**” — che mette insieme quei pezzi. Ecco la scala che stiamo salendo.

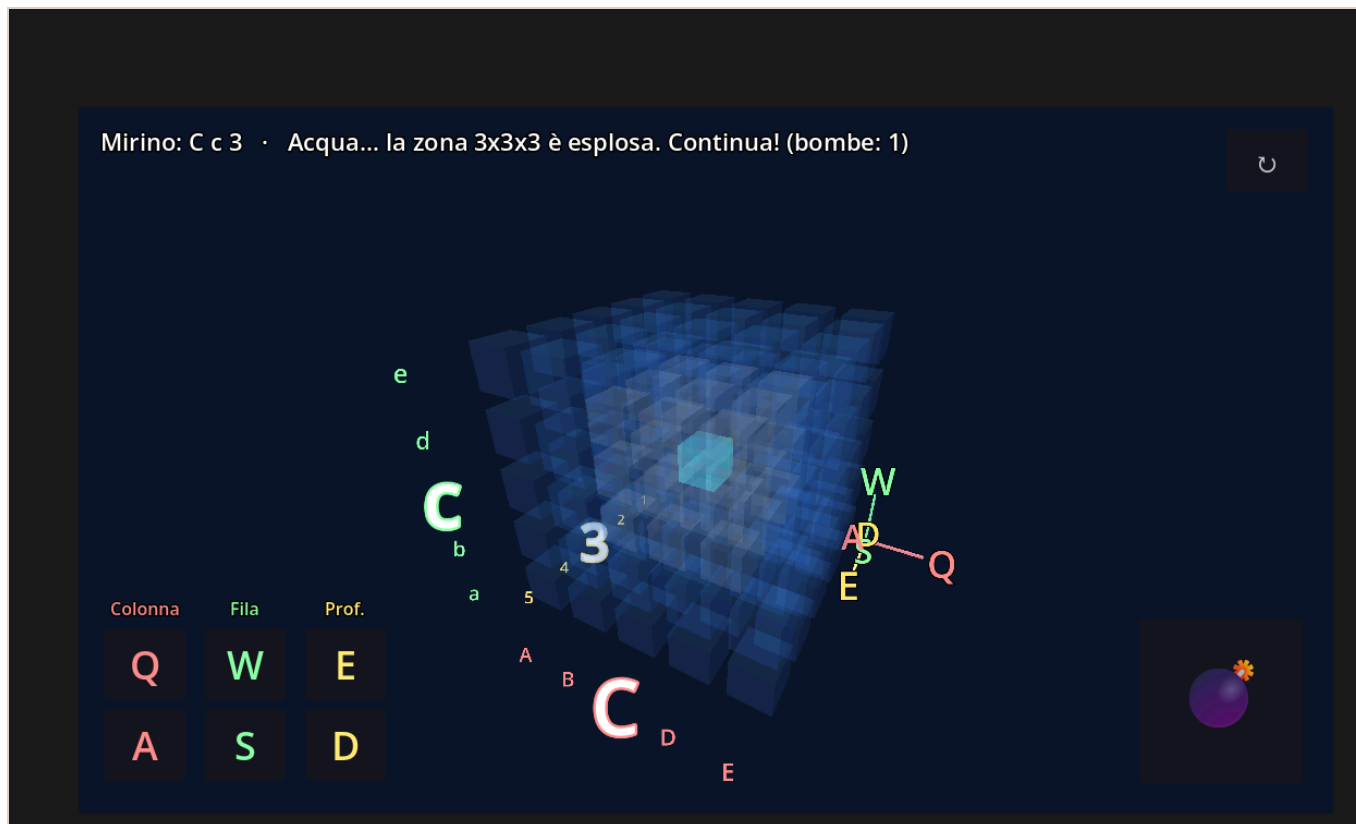
I gradini piccoli: gli esercizi dell’eserciziario

| Esercizio | Cosa impari | Il concetto sotto |
|---------------------------|-------------------------------|---|
| 1 • Il bottone che saluta | Un clic fa succedere qualcosa | Il segnale , il tuo <code>Button1Click</code> di Lazarus |
| 2 • Muovi il quadrato | Far muovere le cose da sole | Il game loop <code>_process(delta)</code> + input |
| 3 • Prendi la moneta | Un mini-gioco vero | Movimento + collisioni + punteggio |
| 4 • Acchiappa le stelle | Un gioco che si può perdere | Vite , stato del gioco e game over |

Ognuno è **corto** e finisce con una **vittoria a schermo**: è fatto apposta così, per vincere subito e non mollare.

Cos’è un “progetto boss”

È un gioco **già pronto**, più grosso, che **non si copia riga per riga**. Si **apre, si gioca e si rende proprio**: cambi i colori, il titolo, ci metti una tua foto. È il **premio**: la cosa figa da mostrare subito. Il primo è “**Affonda la Bonomi**”: lo trovi nell’eserciziario e nella cartella `battaglia-navale-3d/`.



Il "progetto boss" Affonda la Bonomi: una battaglia navale in 3D, dentro un cubo d'acqua.

Dal 2D al 3D: cosa cambia nel boss

Finora abbiamo lavorato in **2D**: due coordinate, **x** orizzontale e **y** verticale, un `Vector2`. Il boss è in **3D**: si aggiunge una terza coordinate, **z**, la **profondità**, un `Vector3`. Il campo di gioco non è più una griglia piatta ma un **cubo** di celle.

Due idee nuove, ma **niente panico**:

- **Si costruisce tutto da codice**: le celle, le luci, la telecamera e i bottoni, invece che a mano nell'editor: sono sempre gli stessi **nodi**, solo tanti, creati con un ciclo `for`.
- Sono gli **stessi concetti di prima, in grande**: il **game loop** gira il cubo, l'**input** muove il mirino. Chi ha fatto gli esercizi 2 e 3 ha già visto tutto.

Come affrontarlo, un gradino alla volta

Non devi finire il boss tutto in una volta: **sali un gradino alla volta**, e a ogni gradino ti porti a casa una vittoria vera.

1. **Gioca** e scegli la difficoltà: con 4 è facilissimo.
2. **Cambia un colore** dell'acqua o del mirino: basta cambiare un numero nel codice, e l'effetto è immediato.

3. **Metti la tua foto**, o un meme, al posto di quella di partenza.

4. **Cambia il titolo** del gioco.

5. **Leggi una funzione piccola** e prova a spiegare **a voce** cosa fa, per esempio come si muove il mirino.

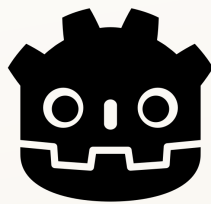
6. Se vai forte: cambia la potenza della bomba o aggiungi un secondo sottomarino.

Parti dal gradino che ti riesce: anche solo giocare e cambiare un colore è già una vittoria da mostrare. Vale sempre: **Vinci subito · Fallo tuo · Mostralo.**

Come useremo l'AI

L'AI è come la **calcolatrice in matematica**: aiuta, ma se non capisci cosa stai facendo non serve a niente.

- Usala per: capire un errore, farti spiegare un concetto, avere un suggerimento, uno **spunto da studiare e modificare**.
- Non usarla per: farti scrivere tutto e consegnarlo senza capirlo.
- **Prova del nove**: se sai **spiegare a voce, riga per riga**, il codice che presenti, la competenza c'è.



GODOT
Game engine

L'Eserciziario

Corso di Godot

Corso a cura del prof. Nicola Regge

Come funziona ogni esercizio

Ogni esercizio ha **4 livelli di aiuto**. Prova sempre da solo, e apri il livello successivo **solo se sei bloccato**:

1.  **Descrizione** — cosa devi ottenere.
2.  **Aiuto** — un indizio su come fare.
3.  **La scena** — quali nodi creare, i “mattoncini”.
4.  **Codice completo** — la soluzione da copiare/incollare.

*Regola: se copi il **Codice completo**, poi devi saper **spiegare a voce cosa fa, riga per riga**. Se lo sai spiegare, hai imparato lo stesso.*

Nel file `.md` i livelli 2–4 sono a scomparsa: clicca sul triangolino per aprirli. Nel PDF sono già aperti.

***Oltre agli esercizi numerati ci sono i “Progetti BOSS”:** giochi già pronti, più grossi, che non si copiano riga per riga — si **aprono, si giocano e si rendono propri**: cambi colori, titolo, ci metti una tua foto. Sono il premio: la cosa figa da mostrare subito agli amici. Il codice è già nel repository, lo capirai un pezzo alla volta.*

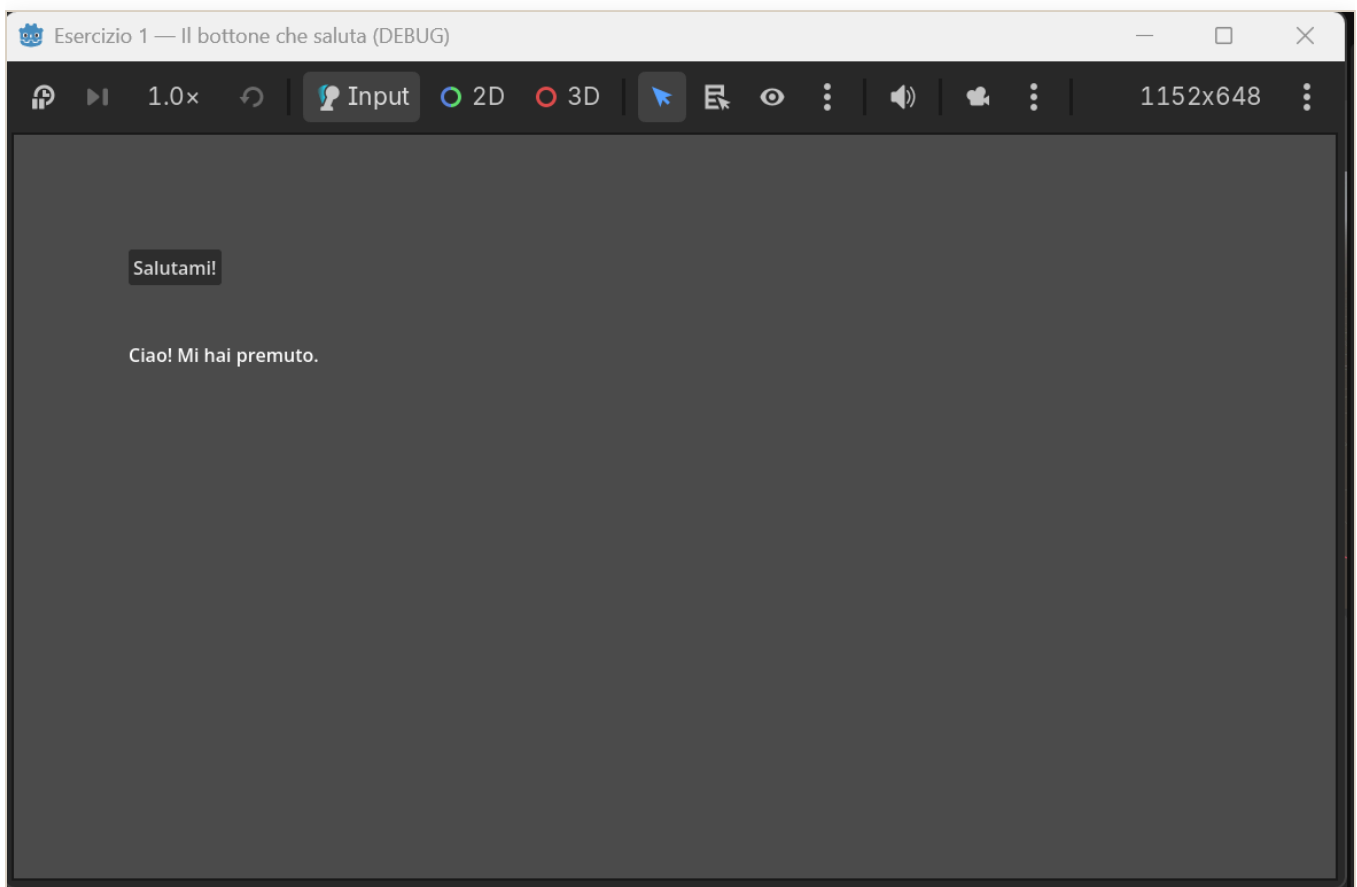
ESERCIZIO 1

Il bottone che saluta

Ponte da Lazarus: è il tuo `Button1Click` che cambia una `Caption`!

Descrizione

Crea una schermata con **un bottone** e **una scritta**. Quando premi il bottone, la scritta deve cambiare, per esempio da “...” a “Ciao! Mi hai premuto”.



Il bottone che saluta: quando lo premi, la scritta cambia.

Fallo tuo: scegli **tu** la frase del saluto e il **colore** della scritta — così il gioco è già tuo. Nessuno lo farà uguale al tuo!

Aiuto

- In Godot il bottone è il nodo **Button**, la scritta è il nodo **Label**.
- La proprietà `text` di Godot è come la **Caption** di Lazarus.

- L'evento "click" in Godot si chiama **segnale** `pressed`. Lo colleghi a una tua funzione con `bottone.pressed.connect(la_mia_funzione)`.

● La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio: **Button** → rinominalo `bottoneScena`.
3. Figlio: **Label** → rinominalo `etichettaScena`.
4. Attacca uno **script** al nodo radice `Main`.

● Codice completo

```
extends Node2D

@onready var bottoneVar: Button = $bottoneScena
@onready var etichettaVar: Label = $etichettaScena

func _ready() -> void:
    # Mettiamo il bottone e la scritta in due punti diversi
    bottoneVar.position = Vector2(100, 100)
    bottoneVar.text = "Salutami!"
    etichettaVar.position = Vector2(100, 180)
    etichettaVar.text = "..."
    # FALLO TUO: cambia il colore della scritta (rosso, verde, blu da 0 a 1)
    etichettaVar.add_theme_color_override("font_color", Color(1, 0, 0))
    # Colleghiamo il click del bottone (il segnale "pressed") alla nostra funzione
    bottoneVar.pressed.connect(_quando_premo)

# Questa è come il tuo Button1Click di Lazarus
func _quando_premo() -> void:
    # FALLO TUO: scrivi qui il TUO saluto
    etichettaVar.text = "Ciao! Mi hai premuto."
```

Guida passo passo — costruiamolo insieme

Cosa costruiamo, e con quali pezzi. Facciamo un bottone che saluta. Servono due pezzi: un **bottone** (quello che l'utente preme) e una **scritta** (dove compare il saluto). L'idea: quando premi il bottone, la scritta cambia. Prima mettiamo i due pezzi; il "quando premi, cambia" lo diremo dopo, nel codice.

Fai i passi in ordine. Non passare al successivo finché il precedente non è a posto.

Passo 1 — Crea il progetto.

1. [APP – Godot], finestra iniziale (il *Gestore progetti*).
2. In alto a **sinistra**, clicca **Crea**.
3. Si apre la finestra per il nuovo progetto. In cima, alla voce **Nome del progetto:**, c'è già scritto **Nuovo progetto di gioco**, tutto evidenziato: scrivi **esercizio1** (sostituisce quello che c'era).
4. Guarda più in basso, alla voce **Renderer:**: clicca il pallino **Compatibilità** (il terzo). Serve per far girare i giochi anche sul web e sui computer della scuola.
5. In basso a destra clicca il bottone **Crea**. (La casella **Modifica ora**, lì vicino, è già spuntata: così l'editor si apre da solo.)

Passo 2 — Entra nel mondo 2D. *Perché:* il nostro gioco è in 2D, quindi lavoriamo nell'ambiente 2D. Se restassi in 3D vedresti una griglia in prospettiva che a noi non serve.

1. In alto al centro, nella fila **2D** **3D** **Script** **Gioco**, clicca **2D**.
2. Ora vedi una tela piatta con dei righelli: è il posto giusto.

Passo 3 — Crea il nodo radice **Main.** *A cosa serve:* è il contenitore principale, il tronco dell'albero; tutti gli altri pezzi staranno dentro di lui.

1. Nel pannello **Scena** (in alto a sinistra), sotto **Crea un nodo radice:**, clicca **Altro nodo**.
2. Si apre la finestra **Crea un nuovo Node**. Nel campo **Cerca:** scrivi **Node2D**: si mette in cima ed è già selezionato.
3. In basso clicca **Crea**.
4. Nel pannello **Scena** compare il nodo **Node2D**. Doppio clic sul nome, scrivi **Main**, premi **Invio**.

Perché due strade? Sotto **Crea un nodo radice:** ci sono anche le scorciatoie **Scena 2D**, **Scena 3D**, **Interfaccia utente**, per i nodi più usati. **Altro nodo** invece apre la finestra con **tutti** i tipi di nodo. Noi useremo sempre **Altro nodo**: così impari l'unica strada che va bene per qualsiasi nodo.

Passo 4 — Aggiungi il bottone **bottoneScena.** *A cosa serve:* è il **bottone** che l'utente cliccherà. Più avanti, nel codice, gli diremo “quando ti premono, chiama la nostra funzione”.

1. Nel pannello **Scena**, clicca **una volta** su **Main** (deve restare evidenziato).
2. In alto a sinistra del pannello, clicca l'icona **+** (*Aggiungi nodo figlio*).
3. Nella finestra, nel campo **Cerca:**, scrivi **Button**: si mette in cima ed è già selezionato.

4. In basso clicca **Crea**.

5. Sotto **Main** compare **Button**. Doppio clic sul nome, scrivi **bottoneScena**, premi **Invio**.

Passo 5 — Aggiungi la scritta **etichettaScena**. A cosa serve: è la **scritta** dove apparirà il saluto; è il pezzo che il giocatore vedrà **cambiare** — l'effetto del gioco.

1. Nel pannello **Scena**, clicca **una volta** su **Main** (deve restare evidenziato lui).

2. In alto a sinistra del pannello, clicca l'icona **+**.

3. Nella finestra, nel campo **Cerca:**, scrivi **Label**: si mette in cima.

4. In basso clicca **Crea**.

5. Doppio clic sul nuovo nodo **Label**, scrivi **etichettaScena**, premi **Invio**.

*Attento alla selezione al punto 1: clicca **Main**, non **bottoneScena** (che hai appena creato nel Passo 4 ed è ancora selezionato). Così la scritta si mette accanto al bottone; se parti da **bottoneScena**, finisce dentro.*

A questo punto, nel pannello **Scena**, l'albero deve essere così:

- **Main**
- **bottoneScena**
- **etichettaScena**

Passo 6 — Attacca lo script a **Main**. A cosa serve: lo script è il file di codice che dà comportamento a **Main**: è lì che scriveremo cosa succede quando premi il bottone.

1. Nel pannello **Scena**, clicca **una volta** su **Main** (deve restare evidenziato).

2. Sempre nel pannello **Scena**, guarda la **barra dei bottoni in alto** (quella con l'icona **+** e la casella **Filtra i nodi**). Subito a destra della casella c'è una piccola icona a forma di **foglio con un + verde**: è il bottone **Allega uno script**. Cliccala.

3. Si apre una finestra per il nuovo script. Non cambiare niente: in basso clicca **Crea**.

4. Si apre l'editor del codice: in cima trovi già scritto **extends Node2D**.

Passo 7 — Metti il codice. Il codice completo è nel riquadro **Codice completo** (livello rosso) qui sopra.

1. **Importante: NON copiarlo dal PDF** — si rovina, i primi caratteri si perdono. Prendilo dal file su GitHub **esercizi/01-bottone-che-saluta/main.gd** (ha il **tasto Copia**), oppure scrivilo a mano: è corto, e così lo capisci meglio.

2. Nell'editor del codice, seleziona tutto con **Ctrl+A** e cancella con **Canc**.

3. Incolla con **Ctrl+V** (o scrivi). Attento ai **rientri con TAB**: in GDScript contano.
4. Salva con **Ctrl+S**.

Passo 8 — Prova.

1. Premi **F5**. La prima volta: se chiede di salvare la scena, dai un nome (per esempio `main`) e clicca **Salva**; se poi chiede la scena principale, clicca **Seleziona corrente**.
2. Si apre la finestra del gioco. Clicca il bottone: la scritta cambia. **Fatto!**

Come funziona, riga per riga

- `extends Node2D` — dice “questo script comanda un nodo Node2D”, il nostro `Main`.
- `@onready var bottoneVar: Button = $bottoneScena` — prende il nodo `bottoneScena` dalla scena e gli dà il soprannome `bottoneVar` (appena la scena è pronta).
- `@onready var etichettaVar: Label = $etichettaScena` — stessa cosa per la scritta.
- `func _ready():` — gira **una volta**, all’avvio.
- `bottoneVar.position = Vector2(100, 100)` — mette il bottone a 100 pixel da sinistra e 100 dall’alto.
- `bottoneVar.text = "Salutami!"` — la scritta sopra il bottone (come la Caption).
- `etichettaVar.position` e `etichettaVar.text = "..."` — mettono la scritta più in basso, con “...” per iniziare.
- `etichettaVar.add_theme_color_override("font_color", Color(1, 0, 0))` — colora la scritta di rosso; i tre numeri sono rosso, verde, blu da 0 a 1: cambiali e cambia il colore.
- `bottoneVar.pressed.connect(_quando_premo)` — collega il **click** del bottone alla nostra funzione: “quando ti premono, chiama `_quando_premo`”.
- `func _quando_premo():` — la nostra funzione, è il tuo `Button1Click` di Lazarus.
- `etichettaVar.text = "Ciao! Mi hai premuto."` — cambia la scritta. Ecco il saluto.

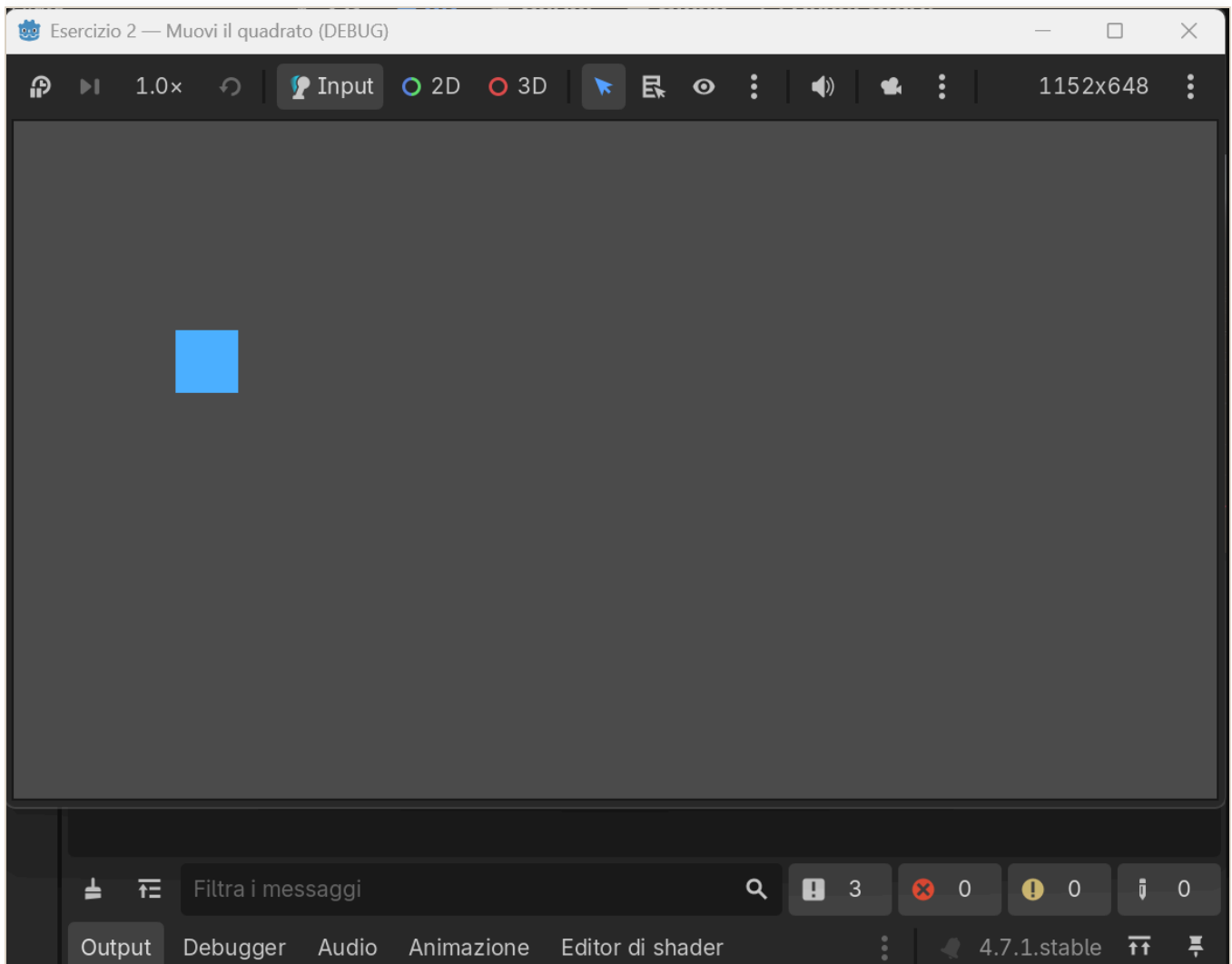
ESERCIZIO 2

Muovi il quadrato

Concetto nuovo: il game loop, cioè' `_process`.

Descrizione

Fai comparire un **quadrato** che puoi muovere in tutte le direzioni con le **freccie** della tastiera.



Il quadrato azzurro che si muove con le frecce.

Aiuto

- Un quadrato colorato semplice = nodo **ColorRect**.

- Il movimento va scritto in `_process(delta)`: e' la funzione che gira ~60 volte al secondo. In Lazarus non c'era: il programma stava fermo.
- Le frecce si leggono con `Input.is_action_pressed("ui_left")`, e allo stesso modo `ui_right`, `ui_up`, `ui_down`.
- Moltiplica sempre la velocita' per `delta`, cosi' va uguale su ogni PC.

● La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio: **ColorRect** → rinominalo `quadratoScena`.
3. Attacca uno **script** al nodo radice `Main`.

● Codice completo

```
extends Node2D

@onready var quadratoVar: ColorRect = $quadratoScena
const VELOCITA: float = 300.0 # pixel al secondo

func _ready() -> void:
    quadratoVar.size = Vector2(60, 60)
    quadratoVar.color = Color(0.3, 0.7, 1.0) # azzurro
    quadratoVar.position = Vector2(200, 200)

# _process gira a OGNI fotogramma: qui muoviamo il quadrato
func _process(delta: float) -> void:
    if Input.is_action_pressed("ui_left"):
        quadratoVar.position.x -= VELOCITA * delta
    if Input.is_action_pressed("ui_right"):
        quadratoVar.position.x += VELOCITA * delta
    if Input.is_action_pressed("ui_up"):
        quadratoVar.position.y -= VELOCITA * delta
    if Input.is_action_pressed("ui_down"):
        quadratoVar.position.y += VELOCITA * delta
```

Guida passo passo — costruiamolo insieme

Cosa costruiamo. Un quadrato che si muove con le frecce. Serve un solo pezzo: un **quadrato**. La novita' è nel codice: il **game loop** `_process`, che gira da solo circa 60 volte al secondo e sposta il

quadrato mentre tieni premuta una freccia.

Passo 1 — Crea il progetto. [APP – Godot], Gestore progetti, in alto a **sinistra** clicca **Crea**. Alla voce **Nome del progetto:** scrivi **esercizio2** (sostituisce quello già scritto). Alla voce **Renderer:** clicca **Compatibilità**. In basso clicca **Crea** (la casella **Modifica ora** è già spuntata).

Passo 2 — Entra nel mondo 2D. In alto al centro clicca **2D** (il gioco è in 2D).

Passo 3 — Crea il nodo radice Main. Nel pannello **Scena**, sotto **Crea un nodo radice:**, clicca **Altro nodo** → nella finestra, campo **Cerca:**, scrivi **Node2D** → **Crea** → doppio clic sul nodo, scrivi **Main**, **Invio**.

Passo 4 — Aggiungi il quadrato quadratoScena. A cosa serve: è il **quadrato** che muoverai, l'oggetto del gioco.

1. Clicca **una volta** su **Main**.
2. In alto a sinistra del pannello, clicca l'icona **+**.
3. Nella finestra, campo **Cerca:**, scrivi **ColorRect** → **Crea**.
4. Doppio clic sul nodo **ColorRect**, scrivi **quadratoScena**, **Invio**.

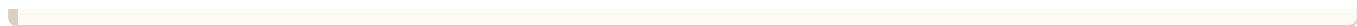
Passo 5 — Attacca lo script a Main. Clicca **una volta** su **Main**. Nella barra dei bottoni in alto del pannello **Scena**, a destra della casella **Filtra i nodi**, clicca l'icona a forma di **foglio con un + verde** (**Allega uno script**) → nella finestra non cambiare niente, clicca **Crea**.

Passo 6 — Metti il codice. Prendi il codice dal file su GitHub **esercizi/02-muovi-il-quadrato/main.gd** (ha il tasto **Copia**), **non dal PDF**. Nell'editor: **Ctrl+A**, **Canc**, **Ctrl+V**, poi **Ctrl+S**.

Passo 7 — Prova. Premi **F5** (se chiede, salva la scena e clicca **Seleziona corrente**). Tieni premute le frecce: il quadrato si muove. **Fatto!**

Come funziona, riga per riga

- `@onready var quadratoVar: ColorRect = $quadratoScena` — soprannome del quadrato.
- `const VELOCITA: float = 300.0` — quanto veloce si muove (pixel al secondo).
- `func _ready():` — all'avvio dà al quadrato dimensione, colore e posizione.
- `func _process(delta):` — **il game loop**: gira a ogni fotogramma, da solo.
- `if Input.is_action_pressed("ui_left"): quadratoVar.position.x -= VELOCITA * delta` — mentre tieni premuta la freccia sinistra, spostalo a sinistra; le altre tre righe fanno lo stesso per destra, su, giù.
- `* delta` — moltiplicare per `delta` fa andare il movimento uguale su ogni PC.



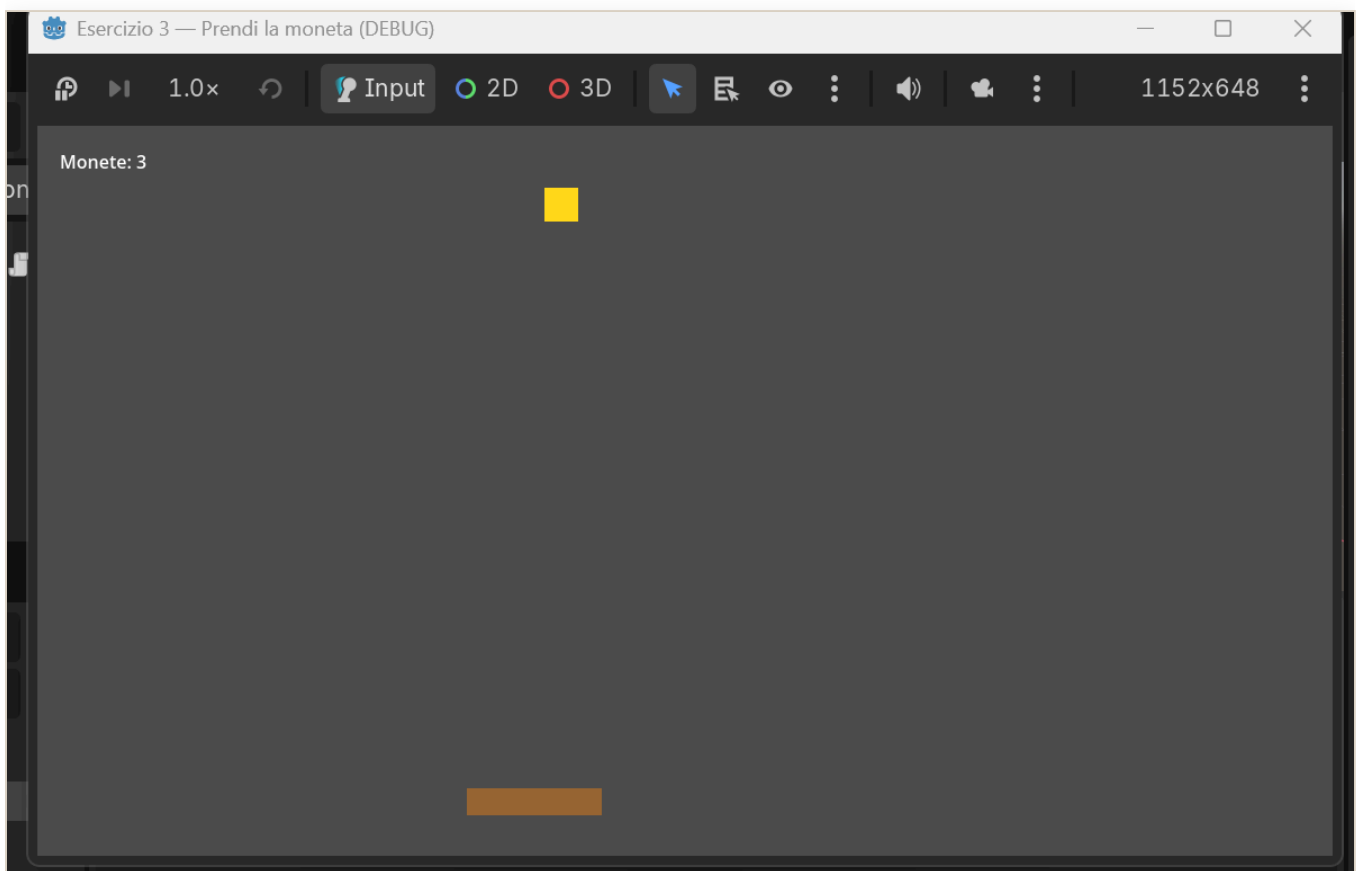
ESERCIZIO 3

Prendi la moneta

Mette insieme: movimento + oggetto che cade + punteggio. Verso il gioco vero.

Descrizione

Un **cestino** in basso, che muovi con le frecce $\leftarrow$ $\rightarrow$, e una **moneta** che cade dall'alto. Se la prendi col cestino fai **+1 punto** e la moneta riparte dall'alto in una colonna a caso. Mostra il punteggio a schermo.



Prendi le monete col cestino: il punteggio sale.

Aiuto

- Cestino e moneta: due **ColorRect**. Il punteggio: un **Label**.
- Muovi il cestino in `_process`, come nell'Esercizio 2 ma solo sinistra/destra.
- Fai scendere la moneta ogni fotogramma: `moneta.position.y += velocita * delta`.
- Per capire se il cestino "tocca" la moneta usa i rettangoli: `Rect2(a.position, a.size).intersects(Rect2(b.position, b.size))`.

- Quando la moneta esce sotto o è presa, rimettila in alto a una ☐ a caso.

● La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio **ColorRect** → `cestinoScena`.
3. Figlio **ColorRect** → `monetaScena`.
4. Figlio **Label** → `punteggioScena`.
5. Attacca uno **script** al nodo radice `Main`.

● Codice completo

```
extends Node2D

@onready var cestinoVar: ColorRect = $cestinoScena
@onready var monetaVar: ColorRect = $monetaScena
@onready var punteggioVar: Label = $punteggioScena

const VELOCITA_CESTINO: float = 500.0
const VELOCITA_MONETA: float = 300.0

var punti: int = 0
var larghezza: float

func _ready() -> void:
    larghezza = get_viewport_rect().size.x
    # Cestino in basso
    cestinoVar.size = Vector2(120, 24)
    cestinoVar.color = Color(0.6, 0.4, 0.2)
    cestinoVar.position = Vector2(larghezza / 2.0 - 60, get_viewport_rect().size.y
    - 60)
    # Moneta
    monetaVar.size = Vector2(30, 30)
    monetaVar.color = Color(1.0, 0.85, 0.1)
    _rimetti_in_alto()
    # Punteggio
    punteggioVar.position = Vector2(20, 20)
    _aggiorna_punteggio()

func _process(delta: float) -> void:
    # Muovi il cestino
    if Input.is_action_pressed("ui_left"):
        cestinoVar.position.x -= VELOCITA_CESTINO * delta
    if Input.is_action_pressed("ui_right"):
        cestinoVar.position.x += VELOCITA_CESTINO * delta
```

```

cestinoVar.position.x = clamp(cestinoVar.position.x, 0, larghezza -
cestinoVar.size.x)

# Fai scendere la moneta
monetaVar.position.y += VELOCITA_MONETA * delta

# Presa?
if Rect2(cestinoVar.position,
cestinoVar.size).intersects(Rect2(monetaVar.position, monetaVar.size)):
    punti += 1
    _aggiorna_punteggio()
    _rimetti_in_alto()
# Persa (uscita sotto)?
elif monetaVar.position.y > get_viewport_rect().size.y:
    _rimetti_in_alto()

func _rimetti_in_alto() -> void:
    var x := randf_range(0, larghezza - monetaVar.size.x)
    monetaVar.position = Vector2(x, -monetaVar.size.y)

func _aggiorna_punteggio() -> void:
    punteggioVar.text = "Monete: %d" % punti

```

Guida passo passo — costruiamolo insieme

Cosa costruiamo. Un cestino che muovi con le frecce ← → e una moneta che cade; se la prendi fai +1 e la moneta riparte dall'alto. Servono tre pezzi: il **cestino**, la **moneta** e il **punteggio** scritto a schermo.

Passo 1 — Crea il progetto. [APP – Godot], Gestore progetti, in alto a **sinistra** clicca **Crea**. Alla voce **Nome del progetto:** scrivi **esercizio3**. Alla voce **Renderer:** clicca **Compatibilità**. In basso clicca **Crea** (la casella **Modifica ora** è già spuntata).

Passo 2 — Entra nel mondo 2D. In alto al centro clicca **2D**.

Passo 3 — Crea il nodo radice **Main**. Pannello **Scena** → **Altro nodo** → **Cerca:** **Node2D** → **Crea** → doppio clic, scrivi **Main**, **Invio**.

Passo 4 — Aggiungi i tre pezzi. Ricorda: **prima di ogni pezzo clicca su** **Main**, così i pezzi restano tutti figli di **Main** (accanto, non uno dentro l'altro).

1. Clicca **Main** → **+** → **Cerca:** **ColorRect** → **Crea** → rinomina in **cestinoScena** (il cestino che muovi).
2. Clicca **Main** → **+** → **Cerca:** **ColorRect** → **Crea** → rinomina in **monetaScena** (la moneta che cade).

3. Clicca **Main** → **+** → **Cerca:** **Label** → **Crea** → rinomina in **punteggioScena** (la scritta del punteggio).

L'albero deve essere: **Main** con sotto **cestinoScena**, **monetaScena**, **punteggioScena**.

Passo 5 — Attacca lo script a Main. Clicca una volta su **Main**. Nella barra dei bottoni in alto del pannello **Scena**, a destra della casella **Filtra i nodi**, clicca l'icona a forma di foglio con un **+** verde (**Allega uno script**) → nella finestra clicca **Crea**.

Passo 6 — Metti il codice. Prendi il codice dal file su GitHub **esercizi/03-prendi-la-moneta/main.gd** (tasto **Copia**), non dal PDF. Nell'editor: **Ctrl+A**, **Canc**, **Ctrl+V**, **Ctrl+S**.

Passo 7 — Prova. Premi **F5** (salva e **Seleziona corrente** se te lo chiede). Muovi il cestino con ← → e prendi la moneta: il punteggio sale. **Fatto!**

Come funziona, riga per riga

- I tre **@onready var ...Var = \$...Scena** — i soprannomi di cestino, moneta e punteggio.
- **func _ready():** — dà dimensioni, colori e posizioni; mette la moneta in alto con **_rimetti_in_alto()**.
- **func _process(delta):** — a ogni fotogramma: muove il cestino con le frecce, fa scendere la moneta, e controlla se è **presa** o **persa**.
- **Rect2(...).intersects(Rect2(...))** — è vero se il cestino e la moneta **si toccano** (presa): +1 punto e moneta di nuovo in alto.
- **elif monetaVar.position.y > ...** — se la moneta esce sotto (persa), la rimette in alto.
- **_aggiorna_punteggio()** — scrive a schermo “Monete: N”.

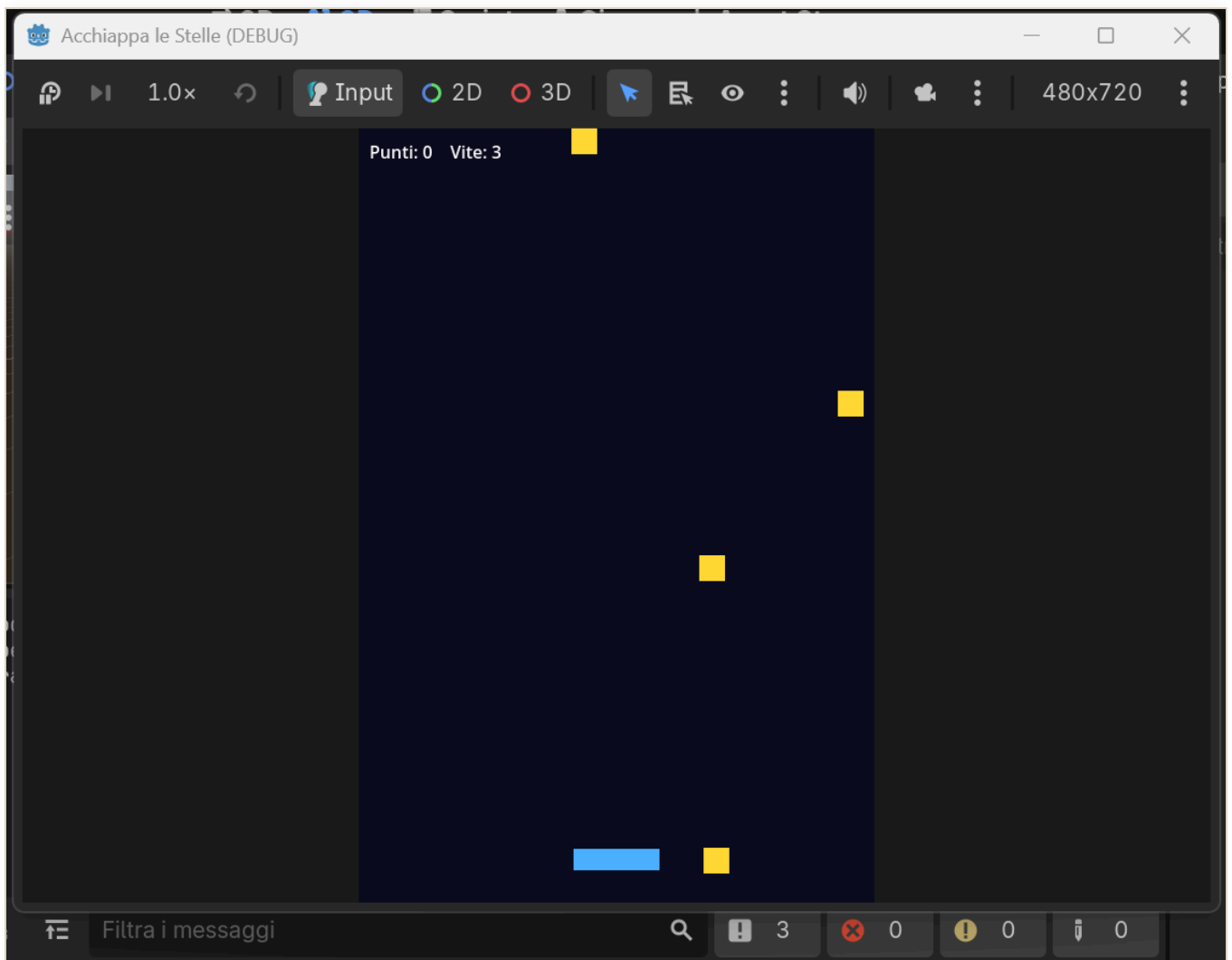
ESERCIZIO 4

Acchiappa le stelle

Concetto nuovo: le vite e il Game Over. Il gioco ora si può perdere e finire.

Descrizione

Una **navetta** in basso, che muovi con le frecce ← →, e una **stella** che cade dall'alto. Se la prendi con la navetta fai +1. Se una stella tocca terra **perdi una vita**. Parti con **3 vite**: quando arrivi a zero è **GAME OVER** e premi **INVIO** per ricominciare. È l'Esercizio 3 che cresce.



La navetta prende le stelle: se ne perdi tre è Game Over.

Aiuto

- È l'Esercizio 3 con altri vestiti: la **navetta** è il cestino, la **stella** è la moneta. Il movimento e la “presa” funzionano allo stesso modo.
- La novità sono due variabili: `vite` (parte da 3) e `in_gioco` (vero/falso).
- Quando una stella esce sotto: `vite -= 1`. Se `vite <= 0`, chiama il Game Over.
- Nel Game Over metti `in_gioco = false` e mostra la Label “GAME OVER”.
- In `_process`, se **non** sei più in gioco, aspetta solo il tasto INVIO (`ui_accept`) per far ripartire tutto da capo.

La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio **ColorRect** → `navettaScena`.
3. Figlio **ColorRect** → `stellaScena`.
4. Figlio **Label** → `hudScena` (punteggio e vite).
5. Figlio **Label** → `gameoverScena` (la scritta di fine partita).
6. Attacca uno **script** al nodo radice `Main`.

Codice completo

```
extends Node2D

@onready var navettaVar: ColorRect = $navettaScena
@onready var stellaVar: ColorRect = $stellaScena
@onready var hudVar: Label = $hudScena
@onready var gameoverVar: Label = $gameoverScena

const VELOCITA_NAVETTA: float = 500.0
const VELOCITA_STELLA: float = 300.0
const VITE_INIZIALI: int = 3

var punti: int = 0
var vite: int = VITE_INIZIALI
var in_gioco: bool = true
var larghezza: float

func _ready() -> void:
    larghezza = get_viewport_rect().size.x
```

```

navettaVar.size = Vector2(90, 20)
navettaVar.color = Color(0.3, 0.7, 1.0)
navettaVar.position = Vector2(larghezza / 2.0 - 45, get_viewport_rect().size.y
- 50)
stellaVar.size = Vector2(28, 28)
stellaVar.color = Color(1.0, 0.85, 0.2)
hudVar.position = Vector2(20, 20)
gameoverVar.position = Vector2(larghezza / 2.0 - 140,
get_viewport_rect().size.y / 2.0 - 20)
gameoverVar.text = "GAME OVER\nPremi INVIO per ricominciare"
gameoverVar.visible = false
_rimetti_in_alto()
_aggiorna_hud()

func _process(delta: float) -> void:
    if not in_gioco:
        if Input.is_action_just_pressed("ui_accept"):
            _ricomincia()
            return
        if Input.is_action_pressed("ui_left"):
            navettaVar.position.x -= VELOCITA_NAVETTA * delta
        if Input.is_action_pressed("ui_right"):
            navettaVar.position.x += VELOCITA_NAVETTA * delta
        navettaVar.position.x = clamp(navettaVar.position.x, 0, larghezza -
navettaVar.size.x)

        stellaVar.position.y += VELOCITA_STELLA * delta

        if Rect2(navettaVar.position,
navettaVar.size).intersects(Rect2(stellaVar.position, stellaVar.size)):
            punti += 1
            _aggiorna_hud()
            _rimetti_in_alto()
        elif stellaVar.position.y > get_viewport_rect().size.y:
            vite -= 1
            _aggiorna_hud()
            _rimetti_in_alto()
            if vite <= 0:
                _game_over()

func _rimetti_in_alto() -> void:
    var x := randf_range(0, larghezza - stellaVar.size.x)
    stellaVar.position = Vector2(x, -stellaVar.size.y)

func _game_over() -> void:
    in_gioco = false
    gameoverVar.visible = true

func _ricomincia() -> void:
    punti = 0
    vite = VITE_INIZIALI
    in_gioco = true

```

```
gameoverVar.visible = false
_rimetti_in_alto()
_aggiorna_hud()

func _aggiorna_hud() -> void:
    hudVar.text = "Punti: %d    Vite: %d" % [punti, vite]
```

Guida passo passo — costruiamolo insieme

Cosa costruiamo. Una navetta che muovi con le frecce ← → e le stelle che cadono: se le prendi fai +1, se ne perdi tre è **Game Over**. La novità è nel codice: le **vite** e il **Game Over** — il gioco ora si può perdere e finire. Servono quattro pezzi: la **navetta**, la **stella**, la scritta di **punti e vite**, e la scritta **GAME OVER**.

Passo 1 — Crea il progetto. [APP — Godot], Gestore progetti, in alto a **sinistra** clicca **Crea**. Alla voce **Nome del progetto:** scrivi **esercizio4**. Alla voce **Renderer:** clicca **Compatibilità**. In basso clicca **Crea** (la casella **Modifica ora** è già spuntata).

Passo 2 — Entra nel mondo 2D. In alto al centro clicca **2D**.

Passo 3 — Crea il nodo radice **Main**. Pannello **Scena** → **Altro nodo** → **Cerca:** **Node2D** → **Crea** → doppio clic, scrivi **Main**, **Invio**.

Passo 4 — Aggiungi i quattro pezzi. Ricorda: **prima di ogni pezzo clicca su** **Main**, così restano tutti figli di **Main**.

1. Clicca **Main** → **+** → **Cerca:** **ColorRect** → **Crea** → rinomina in **navettaScena** (la navetta che muovi).
2. Clicca **Main** → **+** → **Cerca:** **ColorRect** → **Crea** → rinomina in **stellaScena** (la stella che cade).
3. Clicca **Main** → **+** → **Cerca:** **Label** → **Crea** → rinomina in **hudScena** (punti e vite).
4. Clicca **Main** → **+** → **Cerca:** **Label** → **Crea** → rinomina in **gameoverScena** (la scritta di fine partita).

Albero: **Main** con sotto **navettaScena**, **stellaScena**, **hudScena**, **gameoverScena**.

Passo 5 — Attacca lo script a **Main**. Clicca **una volta** su **Main**. Nella barra dei bottoni in alto del pannello **Scena**, a destra della casella **Filtra i nodi**, clicca l'icona a forma di **foglio con un + verde** (**Allega uno script**) → nella finestra clicca **Crea**.

Passo 6 — Metti il codice. Prendi il codice dal file su GitHub **esercizi/04-acchiappa-le-stelle/main.gd** (tasto **Copia**), **non dal PDF**. Nell'editor: **Ctrl+A**, **Canc**, **Ctrl+V**, **Ctrl+S**.

Passo 7 — Prova. Premi **F5** (salva e **Seleziona corrente** se te lo chiede). Muovi la navetta con $\leftarrow$ $\rightarrow$ e prendi le stelle; falne cadere tre per vedere il **GAME OVER**, poi **Invio** per ricominciare. **Fatto!**

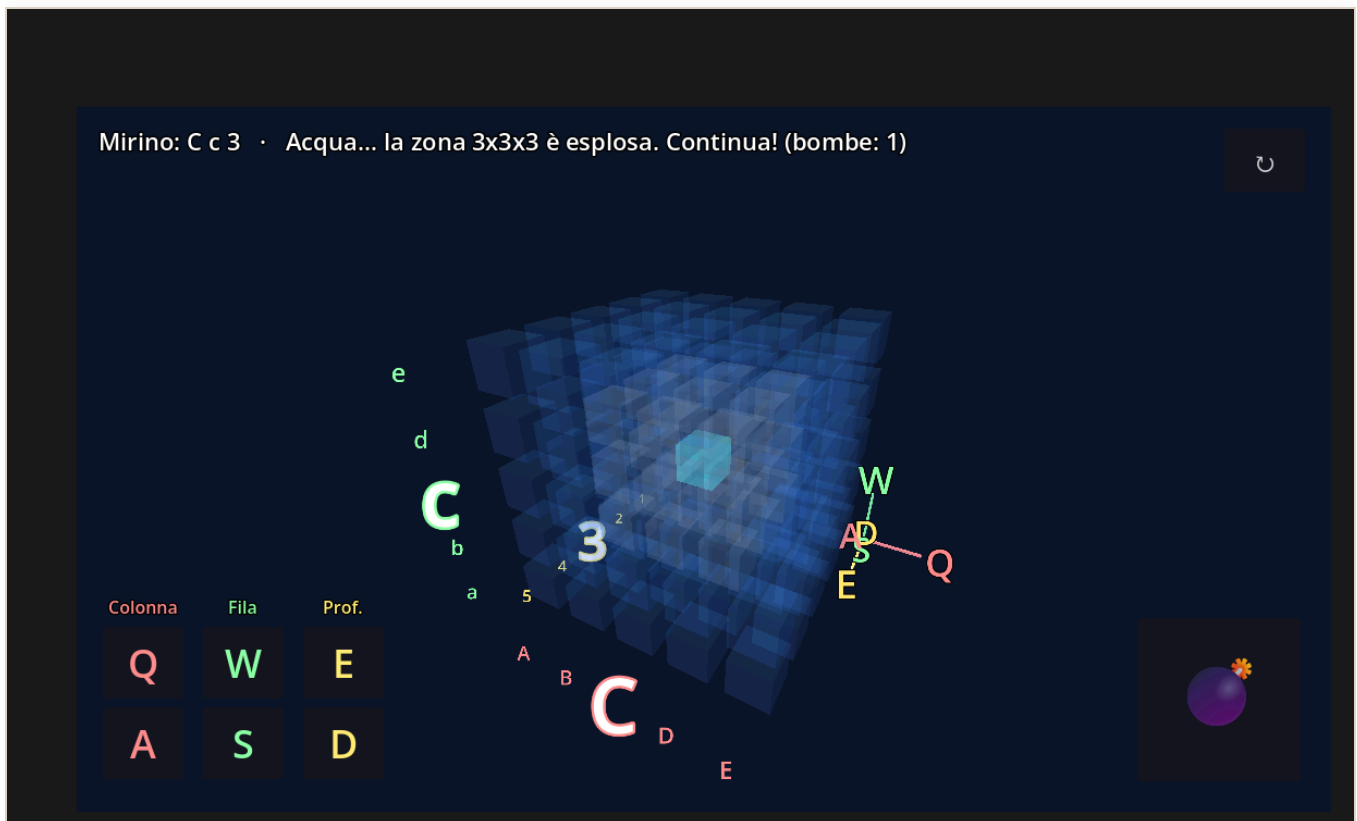
Come funziona, riga per riga

- `var vite := 3` e `var in_gioco := true` — le due novità: quante vite hai, e se la partita è in corso.
- `func _process(delta):` — se **non** sei più in gioco, aspetta solo **Invio** (`ui_accept`) per ricominciare; altrimenti muove la navetta e fa scendere la stella.
- Presa (`intersects`): +1 punto e stella di nuovo in alto.
- Persa (la stella esce sotto): `vite -= 1`; se `vite <= 0` chiama `_game_over()`.
- `_game_over()` — ferma il gioco e mostra la scritta **GAME OVER**.
- `_ricomincia()` — rimette punti e vite a posto e riparte.

Fallo tuo: cambia il colore e la dimensione della navetta e delle stelle, la velocità, o le vite di partenza. Stesso gioco, tema diverso: se al posto delle stelle metti gli organi che cadono dal tavolo, hai il **Chirurgo pasticciatore** (nella cartella `chirurgo-pasticcione/`). Nella cartella `acchiappa-le-stelle/` trovi anche una versione più ricca, con tante stelle insieme, da studiare.

Affonda la Bonomi

Il primo “progetto boss”: una battaglia navale in 3D già giocabile. Si apre, si gioca, si rende proprio.



Affonda la Bonomi in azione: il cubo d'acqua con il mirino verde, le coordinate scritte attorno al cubo e, in basso a sinistra, i comandi colorati dei tre assi (Colonna Q/A, Fila W/S, Profondità E/D).

Descrizione

La solita battaglia navale, ma **in tre dimensioni**: al posto della griglia a righe e colonne c'è un **cubo** di celle d'acqua. Dentro è nascosto **un sottomarino**. Muovi un **mirino** e lanci una **bomba di profondità**: esplode e colpisce una zona **3×3×3** attorno al punto. Se il sottomarino è lì → **COLPITO!** Questo gioco è **già fatto**: il tuo compito è **aprirlo, giocarci e farlo tuo**.

● Aiuto — come aprirlo e giocarci

1. [APP – Godot] finestra iniziale, il Gestore progetti. In alto a **sinistra** clicca **Importa** (o, al centro, **Importa progetto esistente**).

2. Si apre una finestra per cercare i file. Entra nella cartella `battaglia-navale-3d`, clicca sul file `project.godot` per selezionarlo, poi in basso clicca `Apri`.
3. Il progetto viene importato e compare nell'elenco. Fai **doppio clic** sul suo nome per aprirlo (oppure selezionalo e, a destra, clicca `Modifica`).
4. Premi `F5` per eseguire. All'avvio scegli **quanti cubi per lato**, da **4** facilissimo, fino a **10** difficile.
5. Comandi, con le lettere disposte come **tre colonne della tastiera**:
6. **Q / A** = Colonna, in rosso · **W / S** = Fila, in verde · **E / D** = Profondità, in giallo
7. **SHIFT + le stesse lettere** = gira il cubo · **dito/mouse trascinato** = gira il cubo
8. **SPAZIO** = lancia la bomba · **U / INVIO** = rigioca e richiede di nuovo la difficoltà

Fallo tuo — la parte più importante

Apri `battaglia-navale-3d/main.gd` e cambia queste cose per rendere il gioco **tuo**. Dopo ogni modifica premi `F5` e guarda l'effetto:

- **I colori dell'acqua e del mirino:** in alto trovi righe tipo `const COL_ACQUA := Color(...)` e `const COL_MIRINO := Color(...)`. Cambia i tre numeri, rosso verde blu da 0 a 1, e avrai il **tuo** stile.
- **La tua foto al posto di Serena:** metti un file `serena.jpg`, una tua foto o un meme) nella cartella `battaglia-navale-3d/`: comparirà quando affondi il sottomarino, lampeggiando con il teschio dei pirati.
- **Il titolo del gioco:** in `project.godot`, alla voce `config/name="..."`, scrivi il **nome che vuoi tu**.
- **La difficoltà di partenza / la potenza della bomba:** prova a cambiare `RAGGIO_BOMBA`: 1 è la zona 3×3×3, 2 è la zona 5×5×5, molto più potente.

Mostralo: quando l'hai personalizzato, fai una partita davanti a un compagno. “Questo l'ho fatto **io**” vale più di qualsiasi voto.

Il codice completo — dov'è e com'è fatto

Il codice **c'è già tutto** ed è versionato nel repository, nel file `battaglia-navale-3d/main.gd`, circa 600 righe. Non va copiato a mano: è il nostro **progetto boss**, lo leggeremo **un pezzo alla volta**.

Le idee sono le **stesse degli esercizi precedenti**, portate in 3D:

- il **game loop** `_process(delta)` per girare il cubo, come nell'Esercizio 2;
- **leggere i tasti** con `Input.is_key_pressed(...)`, come nel muovere il quadrato;
- **costruire tutto da codice**: celle, luci, telecamera, bottoni, invece che a mano.

Regola d'oro, valida anche qui: se sai **spiegare a voce** cosa fa un pezzo di codice, quel pezzo è tuo. Partiremo dai pezzi più facili, i colori e i comandi, e saliremo piano piano.

Il mio libro di Godot

Questo è il **tu**o libro. Cresce a ogni lezione e a ogni gioco che fai.
A fine anno sarà pieno di cose che hai costruito **tu**.

Nome: _____

Classe: _____

Come si usa (semplice)

- Dopo **ogni lezione** aggiungi una pagina "Lezione".
 - Dopo **ogni esercizio/gioco** aggiungi una pagina "Il mio gioco".
 - Metti sempre uno **screenshot** del tuo lavoro: è la parte più bella. 
 - Alla fine di ogni pagina, prova a **spiegarlo con parole tue**: se lo sai spiegare, l'hai capito davvero.
-

PAGINA — Lezione (copia questo blocco ogni volta)

Lezione del _//_ — *titolo*: _____

Cosa ho imparato oggi (con parole mie):

-

Una cosa nuova che non sapevo:

Screenshot / immagine:

 la mia schermata

PAGINA — Il mio gioco (copia questo blocco per ogni esercizio)

Gioco: _____ (*esercizio n° _*)

Cosa fa il mio gioco:

Cosa ho cambiato per renderlo MIO (colore, nome, personaggio...):

Un problema che ho avuto e come l'ho risolto:

Screenshot del gioco:



il mio gioco

Lo spiego a un amico in 2 righe:

Suggerimento: le immagini mettile in una cartella `immagini/` accanto a questo file, e richiamale come nell'esempio: `![descrizione](immagini/nome.png)`.

03 Scaletta delle prime lezioni

Versione 1.0 — 16/08/2026

Piano di partenza per la Fase 1, quella degli esercizi separati. Serve al prof per avere la rotta pronta: cosa si spiega, in che ordine, con quale esercizio.

Non è una gabbia. Con questi ragazzi il tempo è elastico: una lezione qui può valere una o due ore di classe. Si va al loro passo. La regola sopra ogni cosa resta il motore Vinci subito, Fallo tuo, Mostralo: ogni lezione deve chiudersi con una piccola vittoria concreta e mostrabile.

Ogni esercizio è nell'eserciziario coi quattro livelli di aiuto, e nel manuale trovi il capitolo che lo costruisce passo passo con le foto.

1. Lezione 0, benvenuti in Godot, il colpo d'occhio

1. Obiettivo: far vedere che con Godot si fanno cose fighe, prima ancora di scrivere una riga. Togliere la paura, accendere la voglia.
2. In classe:
3. Aprite Godot portabile, senza installare niente.
4. Fate girare un gioco già pronto, per esempio Affonda la Bonomi o Acchiappa le stelle. Li fai giocare due minuti.
5. Il ponte da Lazarus, detto a voce, senza tecnicismi: in Lazarus c'era la Form, qui c'è la Scena; i componenti diventano i nodi; l'evento del click diventa un segnale. Stessa idea, nome nuovo.
6. La vittoria da mostrare: hanno visto un gioco vero girare, fatto con lo stesso strumento che useranno loro.

2. Lezione 1, il bottone che saluta

1. Materiale: Esercizio 1.
2. Obiettivo: costruire da zero una schermata con un bottone e una scritta che cambia al click.
3. Concetto nuovo, piccolo: nodo, scena, proprietà, segnale.
4. Ponte da Lazarus, diretto:
5. Il bottone è il nodo Button, come il tuo TButton.
6. La scritta è il nodo Label; la sua proprietà text è la vecchia Caption.
7. Il click è il segnale pressed, come il tuo Button1Click.
8. Fallo tuo: ognuno sceglie la frase del saluto e il colore della scritta.

9. La vittoria da mostrare: premo il bottone e compare il mio saluto.

3. Lezione 2, il quadrato che si muove

1. Materiale: Esercizio 2.
2. Obiettivo: muovere un quadrato con le frecce.
3. Concetto nuovo chiave, da introdurre bene: il game loop, cioè `_process(delta)`, che gira da solo circa 60 volte al secondo. La frase da lasciargli: Lazarus reagisce, Godot pulsa. In Lazarus il codice partiva solo quando succedeva qualcosa; qui c'è un battito continuo che possiamo usare per muovere le cose.
4. Fallo tuo: colore, dimensione e velocità del quadrato.
5. La vittoria da mostrare: muovo il mio quadrato sullo schermo con le frecce.

4. Lezione 3, prendi la moneta

1. Materiale: Esercizio 3.
2. Obiettivo: un primo gioco vero: un cestino che si muove, una moneta che cade, un punteggio che sale.
3. Concetti nuovi, piccoli: una variabile che tiene il punteggio, il controllo se due cose si toccano, il far ricomparire la moneta in alto.
4. Fallo tuo: colori, velocità, e cosa cade, una moneta, una stella, una faccina.
5. La vittoria da mostrare: gioco e vedo il punteggio salire quando prendo.

5. Lezione 4, acchiappa le stelle

1. Materiale: Esercizio 4.
2. Obiettivo: trasformare il gioco della moneta in un gioco che si può perdere.
3. Concetto nuovo: le vite e il game over. Il gioco ora ha uno stato, sta giocando oppure è finito, e si può ricominciare premendo INVIO.
4. Fallo tuo: quante vite, l'aspetto della navetta e delle stelle, la scritta di fine partita.
5. La vittoria da mostrare: un gioco completo, con punteggio, vite, game over e ripartenza. Da far provare al compagno di banco.

6. Lezione 5, la prima consegna

1. Materiale: `consegne/INVITO-ALLA-CLASSE.md`.
2. Obiettivo: mettere online il proprio lavoro. È qui che entra Git, nella versione più semplice.

3. Concetto nuovo: la mia copia del corso, il fork, e il commit come salvataggio. Niente branch, niente Pull Request: quelli arrivano in Fase 2, coi gruppi.
4. In classe: il primo giro guidato tutti insieme alla lavagna. Ognuno si fa il fork, copia il modello, compila la scheda con parole sue, carica i file.
5. La vittoria da mostrare: la mia consegna è online, con un link che posso mandare.

7. Lezione 6, personalizza e mostra

1. Obiettivo: consolidare e divertirsi. Chi è avanti personalizza a fondo un gioco o apre un progetto boss; chi è indietro recupera con calma un esercizio.
2. Idea: una piccola gara amichevole di personalizzazioni, si vota il più bello. L'obiettivo non è il voto, è il gusto di mostrare una cosa propria.
3. La vittoria da mostrare: ognuno ha almeno un gioco suo, personalizzato, di cui è fiero.

8. Dopo la Fase 1

1. Quando hanno preso confidenza, si passa alla Fase 2: il progetto di gruppo che cresce, con branch, Pull Request e release. Lì si insegna a lavorare in team, coi ruoli divisi, come in un vero gruppo di sviluppo.

04 Struttura del repository

Versione 1.0 — 16/08/2026

L'albero del repository del corso: cosa c'è e come è organizzato. Serve per capire a colpo d'occhio dove sta ogni cosa, e per far vedere ai ragazzi com'è fatto un vero repository. La mappa descrittiva del materiale sta invece nel documento 02 Indice.

1. Com'è organizzato adesso (colpo d'occhio)

```
corso-godot/
├── 00 CORSO.md / .pdf          quadro generale del corso
├── 01 GLOSSARIO.md / .pdf      termini e sigle spiegati
├── 02 INDICE.md / .pdf         mappa di tutto il materiale
├── 03 SCALETTA-LEZIONI.md / .pdf piano delle lezioni
├── 04 STRUTTURA-REPOSITORY.md questo documento
├── REGOLE-DOCUMENTI.md / .pdf regole di formattazione dei documenti
├── README.md / .pdf           porta d'ingresso del repository
├── CLAUDE.md                  preferenze e contesto del corso
├── RIPRENDIAMO-DA-QUI.md / .pdf nota di lavoro tra una sessione e l'altra
├── CORSO-COMPLETO-vX.Y.pdf    la super-guida: tutti i documenti in un PDF
|
├── manuale/                  i materiali didattici per i ragazzi
|   ├── manuale.md           libro di testo (sorgente)
|   ├── manuale-vX.Y.pdf     libro di testo (PDF versionato)
|   ├── eserciziario.md      esercizi coi 4 livelli di aiuto (sorgente)
|   ├── eserciziario-vX.Y.pdf esercizi (PDF versionato)
|   ├── quaderno-studente-TEMPLATE.md modello del quaderno personale
|   ├── immagini/           gli screenshot richiamati nel libro
|   └── _build/              strumenti che generano i PDF (li usa Claude)
|
├── esercizi/                 progetti Godot pronti, uno per cartella
|   ├── 01-bottone-che-saluta/
|   ├── 02-muovi-il-quadrato/
|   ├── 03-prendi-la-moneta/
|   └── 04-acchiappa-le-stelle/
|
└── consegne/                 kit per raccogliere e correggere i lavori
```

```

|   └─ CREA-ACCOUNT-GITHUB.md
|   └─ INVITO-ALLA-CLASSE.md
|   └─ _MODELLO/                il kit vuoto che i ragazzi copiano
|   └─ _PROF/                   solo prof: come correggere e valutare
|   └─ 2026-2027-1informatica/  un esempio già compilato
|
|   └─ (giochi, un progetto Godot per cartella)
|       └─ battaglia-navale-3d/   Affonda la Bonomi
|       └─ gioco-del-quindici/    il 15 puzzle
|       └─ acchiappa-la-talpa/
|       └─ schiva-gli-asteroidi/
|       └─ rompi-i-mattoni/
|       └─ torta-in-faccia/
|       └─ chirurgo-pasticcione/  primo gioco (backup)
|       └─ acchiappa-le-stelle/   mini gioco di esempio
|
|   └─ docs/                     la cartella pubblicata sul web (GitHub Pages)
|       └─ index.html             Affonda la Bonomi
|       └─ quindici/ talpa/ asteroidi/ mattoni/ torta/   i giochi web
|       └─ cassaforte/           l'app cassaforte
|
|   └─ .github/workflows/
|       └─ deploy-pages.yml       pubblica docs/ sul web a ogni modifica

```

2. Le cartelle, una per una

1. `manuale/` : i materiali didattici. Contiene il libro di testo, l'eserciziario, il modello del quaderno, le immagini e gli strumenti di generazione dei PDF nella sottocartella `_build/`.
2. `esercizi/` : i progetti Godot già pronti della Fase 1, uno per cartella, che i ragazzi aprono e provano.
3. `consegne/` : il kit per la consegna e la correzione dei lavori, con le guide, il modello vuoto, la parte per il prof e un esempio compilato.
4. Le cartelle dei giochi, una per gioco, ognuna un progetto Godot completo con il suo `project.godot`, la scena, lo script e le immagini.
5. `docs/` : la cartella che GitHub Pages pubblica sul web. Dentro ci sono le versioni giocabili dei giochi e l'app cassaforte.
6. `.github/workflows/` : contiene `deploy-pages.yml`, che pubblica `docs/` sul web a ogni modifica del ramo `main`.

3. I documenti in radice

1. I documenti tecnici numerati, da 00 a 04, più le regole di formattazione, il README, la nota di contesto CLAUDE e la nota di lavoro RIPRENDIAMO.
2. La super-guida `CORSO-COMPLETO-vX.Y.pdf` : l'unione di tutti i documenti in un solo PDF, con Sommario e segnalibri.
3. Ogni documento esiste in doppio formato: MD (sorgente) e PDF (generato).

4. Dove si arriverà, il layout obiettivo

1. Il ramo `main` è l'area di lavoro del prof, sempre aggiornata.
2. Le release taggate, come v1.0 e v1.1, sono le versioni congelate e stabili consegnate ai ragazzi.
3. Alla Fase 2, coi progetti di gruppo, si aggiungerà una cartella `progetto-gruppo/` : un gioco che cresce con branch, Pull Request e release, dove i ragazzi imparano a lavorare in team.
4. I ragazzi lavorano su una loro copia, il fork, mai sul repository del prof.

Changelog

| Versione | Data | Cosa è cambiato |
|----------|------------|---|
| 1.0 | 16/08/2026 | Prima stesura: albero del repository attuale e layout obiettivo della Fase 2. |

Consegne — come sono organizzate

Qui finiscono le **consegne dei ragazzi**. Ogni consegna è una cartella con la scheda compilata, il codice e gli screenshot. Da qui il prof (con Claude) corregge, dà un voto e aggiunge una pagina al **manuale personale** dello studente.

Struttura delle cartelle (per gestire più classi e più anni)

```
consegne/  
  _MODELLO/                                ← il KIT VUOTO da copiare (non si tocca)  
    scheda.md  
    ISTRUZIONI.md  
    immagini/  
  
  2026-2027-1informatica/                  ← anno scolastico + classe  
    rossi-mario/                           ← lo studente (cognome-nome)  
      es1-bottone/                         ← la consegna (un esercizio o un titolo)  
        scheda.md  
        main.gd  
        immagini/  
      es2-quadrato/  
      ...  
    bianchi-lucia/  
    ...
```

Così ogni anno si crea una nuova cartella `ANNO-CLASSE/` e tutto resta in ordine nel tempo.

Chi lavora dove (decisione presa)

Ogni ragazzo lavora su una SUA copia del corso (un *fork*), mai su questo repository. Così nessuno può toccare il lavoro degli altri, e il repo del corso resta intatto. Nella sua copia il ragazzo mette le consegne sotto `consegne/`, per esempio `consegne/es1-bottone/`.

Questa cartella `consegne/ANNO-CLASSE/cognome-nome/` qui nel repo del corso è invece l'**archivio del prof**: qui il prof (con Claude) raccoglie le consegne corrette e tiene i **manuali personali** degli studenti.

Per il primo giro con la classe

La guida da dare ai ragazzi la prima volta è `INVITO-ALLA-CLASSE.md` : li porta passo passo dal niente alla prima consegna, tutto da browser. Dalla seconda volta usano `_MODELLO/ISTRUZIONI.md` .

Il flusso (semplice, per iniziare — Fase 1)

1. Una volta sola: il ragazzo si fa la **sua copia** del corso (bottone **Fork** su GitHub, da browser).
2. Per una consegna: **copia** `_MODELLO/` nella sua copia, la rinomina (es. `consegne/es1-bottone/`), **compila** `scheda.md` , ci mette `main.gd` e gli screenshot in `immagini/` .
3. **Carica** i file da browser (`Add file` → `Upload files`) e fa **Commit**.
Niente branch: un semplice salvataggio nella sua copia basta.
4. Manda al prof il **link** della sua consegna. Claude corregge, si decide il voto (numero + breve giudizio) e si aggiunge la pagina al manuale personale.

I branch e le Pull Request li introdurremo in **Fase 2** (progetto di gruppo), quando i ragazzi avranno preso confidenza: un passo alla volta.

Un esempio già compilato, da far vedere ai ragazzi

Dentro `2026-2027-1informatica/rossi-mario/es1-bottone/` c'è una consegna finta ma completa. Serve come modello: i ragazzi la guardano e capiscono com'è fatta una buona consegna. Contiene la `scheda.md` compilata, il `main.gd` con il suo tocco, uno screenshot in `immagini/` e la `valutazione.md` del prof, così si vede tutto il giro, dalla consegna al voto.

La cartella `_PROF/` — solo per il prof

`_PROF/` non la copiano i ragazzi. Dentro c'è come si corregge, la scala del voto e il template della valutazione:

- `_PROF/COME-CORREGGO.md` — il giro completo, i quattro segnali che guardiamo e la scala del voto da rivedere insieme.
- `_PROF/valutazione-MODELLO.md` — il modulo vuoto della valutazione, da copiare nella cartella della consegna col nome `valutazione.md` .

Crearsi l'account GitHub

Guida per i ragazzi: farsi il proprio account su GitHub, il sito dove teniamo il corso e le consegne. Si fa una volta sola. Tutto da browser, niente da installare.

Serve una cosa sola prima di iniziare: un **indirizzo email** a cui puoi accedere (per confermare l'iscrizione). Se non ce l'hai, chiedi al prof come fare.

Passo 1 — Apri il sito

1. `[BROWSER]` Clicca nella barra dell'indirizzo, in alto.
2. Scrivi questo e premi **Invio**:

github.com

Passo 2 — Inizia l'iscrizione

1. `[BROWSER – pagina di GitHub]` In alto a destra clicca **Sign up** (vuol dire "Iscriviti").
2. Si apre una schermata che ti chiede i dati, uno alla volta.

Passo 3 — Metti i tuoi dati

1. **Email**: scrivi il tuo indirizzo email, poi clicca **Continue**.
2. **Password**: inventane una (sceglila robusta, con lettere e numeri) e clicca **Continue**.
3. **Username**: è il tuo nome sul sito. Scegline uno che ti piace; se è già preso, il sito te lo dice e ne provi un altro. Poi **Continue**.
4. Se ti chiede se vuoi ricevere email di novità, rispondi come preferisci (va bene anche di no) e **Continue**.

Passo 4 — Dimostra che non sei un robot

1. Compare un piccolo gioco-quiz (immagini da girare o da scegliere): risolvillo seguendo le istruzioni a schermo.
2. Alla fine clicca il pulsante per creare l'account.

Passo 5 — Conferma l'email

1. `[BROWSER]` GitHub ti manda un **codice** via email.
2. Apri la tua **casella email**, trova il messaggio di GitHub e leggi il codice.
3. Torna su GitHub e scrivi il codice dove te lo chiede.

Fatto: hai il tuo account. Da qui in poi, quando entri su GitHub, sei **tu**.

Nota per il prof

- GitHub richiede un'età minima (13 anni) e un'email per ogni ragazzo. Conviene decidere prima come gestire le email (personali o della scuola) e, se serve, preparare gli account con calma il primo giorno in laboratorio.
- Questa guida serve solo a farsi l'account. Il primo giro di consegna (fork, copia del modello, caricamento) è nella guida `INVITO-ALLA-CLASSE.md`.

Il tuo primo giro — benvenuto

Da oggi hai una **copia tutta tua** del corso. Ci metti i tuoi esercizi, i tuoi giochi, le tue foto. Alla fine dell'anno avrai un quaderno tuo, di cui essere fiero. Qui non si sbaglia "e basta": si prova, si aggiusta, si mostra.

Questo foglio ti porta dal niente alla tua **prima consegna**. Un passo alla volta. Se ti blocchi, chiama il prof: il primo giro lo facciamo insieme.

Cosa ti serve

- Un **browser** sul computer, per esempio Chrome o Edge.
- Il tuo **account GitHub**. Se non ce l'hai, il prof ti dice come si crea.
- L'**indirizzo del corso**, che è questo:

```
https://github.com/nicolaregge-pulse/corso-godot
```

Passo 1 — Apri il corso

1. `[BROWSER]` Clicca nella barra dell'indirizzo, in alto.
2. Scrivi l'indirizzo del corso qui sopra e premi **Invio**.
3. Si apre la pagina del corso, con l'elenco delle cartelle.

Passo 2 — Fatti la tua copia, il fork

Copia tua, in inglese, si dice **fork**. Serve perché nessuno tocchi il tuo lavoro e tu non tocchi quello degli altri.

1. `[BROWSER – pagina del corso]` Guarda **in alto a destra**.
2. Clicca il bottone **Fork**.
3. Nella schermata che compare, clicca **Create fork**.
4. Aspetta qualche secondo.

Passo 3 — Controlla di essere nella TUA copia

1. `[BROWSER]` Guarda **in alto a sinistra**, il nome della pagina.
2. Deve iniziare col **tuo nome utente**, poi `/corso-godot`.
3. Se c'è il tuo nome, sei nel posto giusto. Lavora sempre qui.

Passo 4 — Guarda com'è fatta una consegna

Prima di fare la tua, guarda un esempio già pronto.

1. [BROWSER – la tua copia] Nell'elenco delle cartelle clicca **consegne**.
2. Clicca **2026-2027-1informatica**, poi **rossi-mario**, poi **es1-bottone**.
3. Apri **scheda.md** e leggi: è così che si racconta cosa hai fatto.
4. Torna indietro e apri **_MODELLO**, poi **ISTRUZIONI.md**: spiega tutto con calma.

Passo 5 — Prepara la tua cartella sul computer

Adesso prepari **tre file** dentro una cartella, sul tuo computer.

1. [APP – Esplora file] Crea una cartella nuova sul Desktop.
2. Chiamala **es1-bottone**.
3. Dentro metti il tuo **main.gd**, che sta nella cartella del tuo progetto Godot.
4. Dentro metti anche il tuo **screenshot** del gioco, per esempio **gioco.png**.

Manca la scheda. La scrivi così:

5. [BROWSER – la tua copia] Apri **consegne**, poi **_MODELLO**, poi **scheda.md**.
6. In alto a destra del riquadro del testo clicca l'icona **Copia**.
7. [APP – Blocco note] Apri il Blocco note e incolla con **Ctrl + V**.
8. **Riempi** i campi dopo i due punti e scrivi, con parole tue, cosa fa il gioco.
9. Menu **File** in alto a sinistra, voce **Salva con nome**.
10. Alla voce **Tipo file** scegli **Tutti i file**.
11. Nel nome scrivi esattamente **scheda.md** e salvala **dentro** la cartella es1-bottone.

Attenzione a un tranello: se non scegli **Tutti i file**, il Blocco note salva **scheda.md.txt** e non va bene. Il nome giusto è **scheda.md**, senza altro dopo.

Passo 6 — Carica la cartella nella tua copia

1. [BROWSER – la tua copia] Apri la cartella **consegne**.
2. In alto a destra clicca **Add file**, poi **Upload files**.
3. [APP – Esplora file] Trascina la cartella **es1-bottone** dentro il riquadro della pagina.
4. [BROWSER] In basso, nel riquadro del messaggio, scrivi **consegna es1**.
5. Clicca il bottone verde **Commit changes**.

Passo 7 — Manda il link al prof

1. [BROWSER] Entra nella cartella **consegne**, poi **es1-bottone**.

2. Copia l'indirizzo dalla barra in alto.
3. Mandalo al prof, come vi siete messi d'accordo. Hai consegnato.

Mostralo

Fatto: hai una consegna tua, online, che puoi far vedere. Al compagno, sul telefono, a casa. Il "l'ho fatto io" vale più di qualsiasi voto. Bravo.

Nota per il prof: il primo giro guidalo tu alla lavagna, tutti insieme. Dalla seconda volta i ragazzi useranno da soli `_MODELLO/ISTRUZIONI.md`, che è la versione di riferimento. La correzione e il voto stanno in `_PROF/`.

Come si consegna un esercizio

Leggi queste istruzioni. Se ti blocchi, puoi anche **incollarle a Gemini** e chiedergli aiuto: sa cosa deve contenere la consegna e ti guida.

Prima di tutto: fatti la tua copia del corso (il fork)

Non lavori sul corso del prof: te ne fai una **copia tua** (in GitHub si chiama **fork**), così nessuno tocca il tuo lavoro.

1. Entra su GitHub con il **tuo account** e apri la pagina del corso.
2. In alto a destra clicca **Fork**, poi **Create fork**.
3. Ora in alto c'è il tuo nome: quella è la **tua copia**. Lavora sempre lì.

Cosa deve contenere la tua cartella di consegna

1. **scheda.md** — il modulo compilato (chi sei e cosa hai fatto).
2. **main.gd** — il codice che hai scritto.
3. **immagini/** — i tuoi screenshot (almeno uno del gioco che gira).

Passo per passo

1. **Copia** questa cartella **_MODELLO** nella tua posizione e rinominala col nome dell'esercizio, per esempio **es1-bottone**.
2. Apri **scheda.md** e **riempi i campi** dopo i due punti (cognome, nome, classe, esercizio, data). Poi scrivi, **con parole tue**, cosa fa il tuo programma. Questa parte falla **da solo**: è la prova che hai capito.
3. Metti dentro il tuo **main.gd** e i tuoi **screenshot** nella cartella **immagini/**.
4. **Carica** tutto su GitHub da browser: **Add file** → **Upload files**, trascini i file, poi scrivi un messaggio breve e fai **Commit**.
5. Avvisa il prof che hai consegnato. Fine!

Come viene valutata la tua consegna

Niente sorprese: sai già cosa guardiamo. Sono quattro cose, in questo ordine.

1. **Consegna completa** — c'è la scheda, il codice e almeno uno screenshot.
2. **Funziona** — il gioco parte e fa quello che deve.
3. **Il tuo tocco** — hai cambiato qualcosa di tuo, anche piccolo.

4. **Lo sai spiegare con parole tue** — è la cosa più importante. Se sai raccontare cosa fa il tuo programma, vuol dire che l'hai capito davvero.

Il voto arriva sempre con due righe di **giudizio**: prima una cosa che hai fatto bene, poi il prossimo passo. Se qualcosa non va, non è una bocciatura: ti diciamo esattamente cosa sistemare e puoi **ri-consegnare**. Sbagliare è normale, succede a tutti i programmatori, anche ai professionisti.

Il patto con l'AI (Gemini)

Gemini può aiutarti a **capire** un errore, a spiegarti un pezzo, a sistemare la scheda. **Non** deve scrivere al posto tuo la parte "cosa ho fatto con parole mie": quella è tua, ed è ciò che verrà valutato davvero. L'AI aiuta a capire, non a saltare il pensiero.

Come fare uno screenshot (Windows)

`Win + Shift + S` → trascini il riquadro → clicchi l'avviso in basso a destra → **dischetto (Salva)** → lo salvi nella cartella `immagini/` con un nome semplice (es. `gioco.png`).

Scheda di consegna

- Cognome:
- Nome:
- Classe:
- Esercizio / titolo:
- Data:

Cosa ho fatto (con parole mie)

Il mio tocco (cosa ho cambiato per renderlo mio)

Dove mi sono bloccato (e come ne sono uscito)

Come correggo le consegne — nota per il prof

Questa cartella `_PROF/` è **solo per il prof**. I ragazzi non la copiano e non la compilano: loro usano `_MODELLO/`. Qui c'è come si corregge e si dà il voto.

Il giro completo di una consegna

1. Il ragazzo consegna nella sua copia del corso e ti manda il **link**.
2. Tu lo passi a **Claude**: leggiamo insieme scheda, codice e screenshot.
3. Claude ti prepara una bozza di **valutazione** partendo da `valutazione-MODELLO.md`.
4. Il voto si decide insieme: **l'ultima parola è tua**. Poi la valutazione va nella cartella della consegna, col nome `valutazione.md`.
5. Claude aggiunge una **pagina** al manuale personale dello studente, così il suo quaderno cresce a ogni esercizio.

I quattro segnali che guardiamo

Non si guarda solo se il codice gira. Si guardano quattro cose, nell'ordine del nostro motore Vinci subito, Fallo tuo, Mostralo:

1. **Consegna completa** — c'è la scheda, il codice e almeno uno screenshot.
2. **Funziona** — il gioco parte e fa quello che deve.
3. **Il tuo tocco** — ha cambiato qualcosa di suo, anche piccolo.
4. **Lo sa spiegare con parole sue** — è la prova del nove. Se sa raccontare cosa fa il suo programma, la competenza c'è davvero, anche se un pezzo è copiato.

La scala del voto — approvata

Questa è la scala concordata. Partiamo dall'alto, non dallo zero: chi ci prova davvero parte già da un buon posto. Sul singolo caso l'ultima parola resta al prof.

- **9 - 10** — Completa, funziona, ha un suo tocco e sa spiegarla bene.
- **7 - 8** — Completa e funziona, tocco presente, spiegazione un po' incerta ma sua.
- **6** — Funziona ed è consegnata, ma il tocco è minimo oppure la spiegazione è debole. Si indica con precisione cosa aggiungere.
- **5** — Consegna incompleta o non funziona, ma c'è un tentativo vero. Non è una bocciatura a freddo: si dice **esattamente** cosa manca e si può **ri-consegnare**. L'errore non fa vergogna, si sistema.

Regola sopra ogni cosa: il **giudizio è sempre incoraggiante**. Prima una cosa andata bene, poi un prossimo passo concreto. Questi ragazzi mollano se si sentono scartati: il voto deve dare voglia di rifare, non di sparire.

Il patto con l'AI, lato correzione

La parte **cosa ho fatto con parole mie** va valutata come sua. Se è chiaramente copiata di peso da un AI, non si castiga: si chiede di **raccontarla a voce**, e si valuta quello. Sapere spiegare è la vera misura.

Dove va la valutazione

- Nella cartella della consegna, accanto alla scheda dello studente, come `valutazione.md`. Vedi l'esempio in `consegne/2026-2027-1informatica/rossi-mario/es1-bottone/`.
- Il template vuoto è `valutazione-MODELLO.md`, qui in `_PROF/`.

Valutazione

- Studente:
- Esercizio:
- Data correzione:

Voto

Giudizio breve

I quattro segnali

- Consegna completa, cioè scheda, codice e almeno uno screenshot:
- Funziona, cioè il gioco parte e fa quello che deve:
- Il tuo tocco, cioè ha cambiato qualcosa di suo:
- Lo sa spiegare con parole sue:

Prossimo passo

Scheda di consegna

- Cognome: Rossi
- Nome: Mario
- Classe: 1 Informatica
- Esercizio / titolo: Esercizio 1 - Il bottone che saluta
- Data: 15/10/2026

Cosa ho fatto (con parole mie)

Ho messo sulla scena un bottone e una scritta. All'inizio la scritta fa vedere solo tre puntini. Quando premo il bottone parte una funzione che cambia la scritta e ci mette il mio saluto. In pratica è come il click di Lazarus: premo e cambia la Caption, solo che qui la Caption si chiama text e il click si chiama segnale pressed.

Il mio tocco (cosa ho cambiato per renderlo mio)

Ho fatto la scritta verde invece che rossa e ho cambiato il saluto: adesso dice "Ciao Mario, bella lì!". Ho anche scritto "Premi qui" dentro al bottone.

Dove mi sono bloccato (e come ne sono uscito)

All'inizio non capivo il colore: ho scoperto che i numeri vanno da 0 a 1 e non da 0 a 255. Ho messo `Color(0, 1, 0)` e la scritta è diventata verde. Il resto liscio.

Valutazione

- Studente: Mario Rossi, 1 Informatica
- Esercizio: Esercizio 1 - Il bottone che saluta
- Data correzione: 16/10/2026

Voto

9

Giudizio breve

Bravo Mario. La cosa più bella è che hai capito da solo il colore che va da 0 a 1 e l'hai scritto con parole tue: quello è saperlo spiegare davvero. Il gioco parte, il saluto è tuo e si vede che ti sei divertito. Prossima volta prova a mettere due saluti diversi che cambiano a turno: sei pronto.

I quattro segnali

- Consegna completa, cioè scheda, codice e almeno uno screenshot: sì, c'è tutto.
- Funziona, cioè il gioco parte e fa quello che deve: sì, la scritta cambia al click.
- Il tuo tocco, cioè ha cambiato qualcosa di suo: sì, colore verde, saluto tuo e scritta nel bottone.
- Lo sa spiegare con parole sue: sì, e anche bene, soprattutto il colore da 0 a 1.

Prossimo passo

Far cambiare il saluto a turno, un click una frase, il click dopo un'altra.

Esercizi — soluzioni funzionanti

Qui c'è, per ogni esercizio dell'eserciziario, un **progetto Godot già pronto e funzionante**: la versione "fatta", da aprire e provare subito.

| Cartella | Esercizio | Cosa mostra |
|------------------------|-------------|--|
| 01-bottone-che-saluta/ | Esercizio 1 | Un clic sul bottone cambia una scritta. Il segnale, cioè l'evento click. |
| 02-muovi-il-quadrato/ | Esercizio 2 | Un quadrato che si muove con le frecce. Il game loop <code>_process</code> . |
| 03-prendi-la-moneta/ | Esercizio 3 | Un cestino prende le monete che cadono, con punteggio. |

Il **Progetto BOSS** (Affonda la Bonomi) ha una sua cartella a parte:
battaglia-navale-3d/.

Come si apre un esercizio

1. [APP — Godot] nella finestra iniziale, il Gestore progetti, in alto a destra clicca **Importa**.
2. Scegli la cartella dell'esercizio (per esempio 01-bottone-che-saluta) e il file **project.godot**, poi **Importa e modifica**.
3. Premi **F5** per giocare.

Nota per il docente: questi sono i progetti-soluzione. Per la classe, i ragazzi partono dall'eserciziario e provano da soli i 4 livelli d'aiuto; questi progetti servono a te per mostrare il risultato o per correggere al volo.

Affonda la Bonomi (battaglia navale 3D — prototipo)

Una battaglia navale **in tre dimensioni**: invece della solita griglia a righe e colonne, c'è un **cubo** di celle d'acqua. Dentro è nascosto **un sottomarino**. Lanci una **bomba di profondità** su una cella: esplode e colpisce una zona **3×3×3** attorno al punto. Se il sottomarino è in quella zona → COLPITO!

 È una **prima bozza** fatta per Nicola, da provare e migliorare insieme (grafica, effetti, difficoltà). Non è ancora un esercizio per i ragazzi.

All'avvio: scegli la difficoltà

Appena parte, il gioco chiede **quanti cubi per lato** vuoi: da **4** (facilissimo, comodo per provare) a **10** (difficile). Più grande è il cubo, più celle e più nascondigli per il sottomarino. Clicca un numero e la partita comincia. Ogni volta che **rigiochi** (INVIO o il tasto **↵**) la domanda ricompare, così puoi cambiare difficoltà a ogni partita.

Coordinate di ogni cella

- **Colonna** = LETTERA MAIUSCOLA (A B C D E ...)
- **Fila** = lettera minuscola (a b c d e ...)
- **Profondità** = NUMERO (1 2 3 4 5 ...)

Le lettere e i numeri sono scritti **attorno al cubo** per ritrovarti.

Come si gioca (6 tasti + Shift)

Il **mirino** è la cella che brilla di verde. Lo muovi con **sei lettere**, disposte come le **tre colonne della tastiera QWERTY** (il tasto di sopra = "+", quello di sotto = "-"), ognuna del **colore del suo asse** (le vedi anche sulle frecce accanto al cubo):

- **Q / A** = **Colonna** (+/-) → lettere **rosse**
- **W / S** = **Fila** (+/-) → lettere **verdi**
- **E / D** = **Profondità** (+/-) → lettere **gialle**
- **SHIFT + le stesse lettere** = **girano il cubo** invece di muovere il mirino (così bastano 6 tasti: lo Shift attiva/disattiva la rotazione).
- **Mouse trascinato** = gira il cubo (in più, se vuoi).
- **SPAZIO** = lancia la bomba di profondità sulla cella del mirino (zona 3×3×3).
- **INVIO** = rigioca: torna alla domanda "quanti cubi per lato?" e riparte.

Quando colpisci 💣

Se la bomba prende il sottomarino, esplode e a tutto schermo si **alternano lampeggiando** la faccia di **Serena** e un **teschio con le ossa** (💀, la bandiera dei pirati), della stessa dimensione; poi l'overlay sparisce e si vede il **sottomarino affondato** (inclinato). Per far comparire la foto basta mettere un file **serena.jpg** nella cartella **battaglia-navale-3d/** (se non c'è, resta solo il teschio).

Sul telefono/tablet (touch)

In basso a sinistra ci sono **tre colonnine di bottoni colorate**, una per asse (come sulla tastiera): **Colonna** (rosso, Q/A), **Fila** (verde, W/S), **Profondità** (giallo, E/D) → il bottone **sopra** muove in "+", quello **sotto** in "-", così sai sempre quale asse muovi. Le **tre frecce colorate** accanto al cubo mostrano le direzioni degli assi (con su scritte le lettere-tasto) e girano insieme al cubo. Il 💣 grande in basso a destra lancia la bomba, il 🔄 rigioca. Per **girare il cubo** trascini il **dito**.

Come portarlo sul telefono (via Web, senza installare niente)

1. [APP – Godot] menù **Progetto** → **Esporta...**
2. **Aggiungi...** → **Web**. Se chiede i *modelli di esportazione*, clicca per **scaricarli** (una volta sola).
3. **Esporta progetto** → scegli una cartella (es. **web/**) e come nome file **index.html** → esporta.

4. Quei file vanno messi **online**: la via più semplice e browser-only è **GitHub Pages** (Impostazioni del repo → *Pages*). Poi apri il **link** sul telefono. → Nessuna installazione, giochi dal browser e puoi condividerlo.

Quando punti una cella, la sua **tripletta di coordinate** (attorno al cubo) diventa **grande e bianca**, così vedi subito cosa hai selezionato.

Come si apre (per Nicola)

1. [APP – Godot] nella finestra iniziale (il *Gestore progetti*), in alto a destra clicca **Importa**.
2. Vai nella cartella **battaglia-navale-3d** e scegli il file **project.godot**, poi **Importa e modifica**.
3. Quando l'editor è aperto, in alto a destra premi ► (Esegui il progetto) — oppure il tasto **F5**.

Cosa possiamo migliorare (idee)

- Sottomarino più "vero" (colori, luci), bolle d'acqua, suono dell'esplosione.
- Più sottomarini quando questo funziona bene.
- Un contatore dei tentativi migliori (record) e magari un timer.

Gioco del Quindici con la foto

Il classico **gioco del quindici**, ma le tessere sono i **pezzi di una foto**:
l'obiettivo è rimetterle in ordine e **ricomporre l'immagine**. Le tessere hanno
una cornice in stile **legno**.

 **Prima bozza** da provare e migliorare (idea di Nicola). Non è ancora un
esercizio per i ragazzi.

Come si gioca

1. All'avvio scegli la **foto** — la tua (`Scegli dal computer...`) oppure quella
di **esempio** — e la **difficoltà**: `3 × 3` (facile) o `4 × 4` (classico).
2. Una tessera manca: resta un **buco**. Clicca una tessera **vicina al buco** e
ci scivola dentro.
3. Quando tutte le tessere sono al posto giusto, la **foto è ricomposta**: hai vinto.
 -  **Rimescola** = nuova partita · **Menu** = torna alla scelta di foto e difficoltà.
 - In alto a destra c'è l'**anteprima** ("Modello"): la foto intera in piccolo, per
sapere dove va ogni pezzo.
 - L'interruttore **Mostra i numeri** accende/spegne i numeri sulle tessere: aiuto
per chi è in difficoltà, sfida in più per chi lo tiene spento.

Come si apre (per Nicola)

1. `[APP – Godot]` finestra iniziale, in alto a destra **Importa** .
2. Scegli la cartella `gioco-del-quindici` e il file `project.godot` ,
poi **Importa e modifica** .
3. Premi `F5` per giocare.

Cosa possiamo ancora migliorare

- Vera **texture di legno** (una foto di legno) al posto del marrone finto.
- Suono quando una tessera scivola.
- Un **timer** e il record delle mosse.

Già fatti: anteprima "Modello", interruttore mostra-numeri, festa alla vittoria.

Nota tecnica

La foto viene tagliata con `AtlasTexture` (ritaglia un pezzo di immagine). Il mescolamento si fa con tante **mosse casuali valide**, così il puzzle è **sempre risolvibile** (un mescolamento del tutto casuale a volte non lo è).

Regole di formattazione dei documenti — Corso Godot

Versione 1.0 — 16/08/2026

Regole per come si scrivono e si consegnano i documenti del corso Godot. Questo documento segue le regole che descrive. Fonte originale: regole del sistema documenti; qui adattate al corso Godot (le parti specifiche del progetto Quidoo Pulse, come STATO_PRODOTTO, Guido e VPS, non si applicano e sono segnalate).

1. Ambito: due categorie di documenti

1. Documenti tecnici e di servizio: 00 Corso, 01 Glossario, 02 Indice, 03 Scaletta, le guide del kit consegne, i README, questo documento. Seguono lo standard pieno qui sotto. (RIPRENDIAMO-DA-QUI resta una nota di servizio interna, non numerata.)
2. Materiali didattici per i ragazzi: manuale, eserciziario, quaderno dello studente. Tengono il loro stile pedagogico (semplice, accogliente, con incoraggiamento), come imposto dal CLAUDE.md del corso. Di questo standard seguono solo le parti compatibili: doppio formato MD + PDF (regola 2), versione nel nome file (regola 10), termine con spiegazione (regola 8), e la REGOLA 0 (regola 11). Non si applicano invece: niente elenchi puntati (regola 6) e niente emoji (regola 7), perché lo stile amichevole e i pallini colorati dei livelli di aiuto dell'eserciziario servono davvero.

2. Formati e consegna

1. Ogni documento esiste sempre in due forme: MD (sorgente versionata) e PDF (generato dall'MD).
2. Consegna doppia: un PDF combinato, la super-guida, come stampa unica; e i singoli documenti numerati.
3. Il combinato del corso è `CORSO-COMPLETO`, generato da `manuale/_build/volume_pdf.py`. Ogni nuovo documento entra nel combinato, oltre a quelli già elencati.

3. Titoli

1. Numero d'ordine nel titolo e nel nome file: 00, 01, 02, 02b, e così via.
2. Nella numerazione niente trattino: si scrive `02 Panoramica`, non `02 – Panoramica`.

3. Titoli mai orfani: un titolo sta sulla stessa pagina dell'inizio del suo contenuto o della sua tabella, mai in fondo a una pagina con il contenuto nella pagina dopo.

4. Indice e navigazione

1. INDICE è la guida di lettura: elenco dei documenti, ognuno con una riga di descrizione.
2. La super-guida ha due indici: un Sommario stampato in testa, con documento e numero di pagina, e i segnalibri PDF cliccabili nella barra laterale. Entrambi si rigenerano da soli a ogni generazione del combinato.

5. Font e numerazione delle sezioni

1. Font unico nei PDF: DejaVu.
2. La numerazione delle sezioni è automatica e gerarchica.

6. Liste

1. Nei documenti tecnici e di servizio: solo liste numerate e gerarchiche, nella forma 1, poi 1.1, poi 1.1.2. Niente elenchi puntati.
2. Nei materiali didattici per i ragazzi: gli elenchi puntati restano ammessi, perché più leggeri e amichevoli.

7. Caratteri decorativi ed emoji

1. Nei documenti tecnici e di servizio: pochi o nessun emoji decorativo. Il generatore converte gli emoji in testo, per esempio [CRITICO], [OK], [ATTENZIONE], oppure li rimuove.
2. Per evidenziare si usano i box colorati semantici della legenda (sezione 9), non le faccine.
3. Nei materiali didattici per i ragazzi: gli emoji con funzione restano, in particolare i pallini dei livelli di aiuto dell'eserciziario, perché il generatore li usa per colorare i livelli.

8. Sigle, acronimi e punteggiatura

1. Ogni sigla con la forma esplicita tra parentesi alla prima occorrenza: VPS (Virtual Private Server: server privato virtuale).
2. I termini ricorrenti stanno nel glossario.
3. Termine con spiegazione: due punti, come in Git: sistema di versionamento.

4. Inciso a metà frase: virgole se leggero, parentesi se accessorio, trattino lungo solo per uno stacco forte voluto.
5. Notazione numerica italiana: virgola per i decimali, punto per le migliaia. Date nel formato giorno/mese/anno nel testo, anno-mese-giorno nei dati e nei database.

9. Box colorati: legenda canonica

1. Rosso, DISALLINEAMENTO: qualcosa che stride con la realtà o con la fonte di verità, cioè un errore noto da correggere.
2. Blu, DA CONFERMARE o IN ATTESA: la direzione verso cui si converge ma senza ufficialità. Quando arriva la conferma, il box sparisce.
3. Giallo, NOTA: semplice nota esplicativa, non segnala problemi.

10. Versioning

1. Una versione stampata è congelata: da lì non si tocca più.
2. Correzioni e aggiunte vanno nella versione successiva, registrate nel changelog della versione. Ogni versione stampata può avere una errata in coda al combinato.
3. Ogni documento porta la sua versione: un documento nuovo parte da 1.0. La super-guida ha la versione della raccolta.
4. Il numero di versione sta sempre nel nome del file PDF, per esempio `manuale-v0.20.pdf`, e coincide con la versione scritta nell'intestazione dell'MD.

11. Fonte di verità

1. Nel corso Godot la fonte di verità dei contenuti è il manuale, e la mappa di tutto il materiale è il documento 02 Indice.
2. La regola del progetto Quidoo Pulse, per cui il documento 00 STATO_PRODOTTO di Guido è la stella polare e non si riscrive ma si recepisce, riguarda quel progetto e non il corso Godot.

12. Regola per la comunicazione all'utente

1. REGOLA 0, assoluta: tutto ciò che l'utente deve copiare, cioè comandi, URL, email, valori, va in un blocco di codice con il bottone copia, mai in linea nel testo e mai in citazione.

Changelog

| Versione | Data | Cosa è cambiato |
|----------|------------|---|
| 1.0 | 16/08/2026 | Prima stesura: recepite le regole di formattazione e adattate al corso Godot, con la distinzione tra documenti tecnici e materiali didattici. |

Corso Godot

Il corso di **Godot** e **GDScript** per i ragazzi.

Per l'elenco completo e aggiornato di tutto il materiale, vedi [INDICE-DEL-MATERIALE.md](#): manuale, esercizi, giochi, kit di consegna e altro.

Concetti base di Godot (il modello mentale)

Godot organizza tutto con **4 concetti**. Se capisci questi, capisci Godot:

| Concetto | Cos'è | Analogia |
|---------------------------------------|---|------------------------------------|
| Progetto | Una cartella con dentro un file <code>project.godot</code> + tutti i file del gioco | La "cartella del gioco" |
| Nodo
(Node) | Il mattoncino base. Ne esistono di tanti tipi: <code>Sprite2D</code> (immagine), <code>Label</code> (testo), <code>Timer</code> (cronometro), <code>Node2D</code> (contenitore 2D)... | Un pezzo di LEGO |
| Scena
(Scene) | Un albero di nodi salvato in un file <code>.tscn</code> . Un livello, un menu, un personaggio... | Una costruzione di LEGO |
| Script
(<code>.gd</code>) | Codice GDScript attaccato a un nodo, che gli dà comportamento | Le istruzioni "cosa fai quando..." |

Regola d'oro: in Godot **tutto è un nodo**, le scene sono nodi messi insieme ad albero, e gli script danno vita ai nodi.

GDScript in 30 secondi

GDScript è il linguaggio di Godot: assomiglia molto a **Python** (indentazione a TAB, niente punto e virgola). Due funzioni speciali che Godot chiama da solo:

- `func _ready():` → eseguita **una volta**, quando il nodo entra in scena.
- `func _process(delta):` → eseguita **a ogni fotogramma** (~60 volte/sec).
`delta` = secondi passati dall'ultimo fotogramma.

Serve un plugin per iniziare?

No. GDScript, l'editor di codice, il 2D e il 3D sono **già dentro**

Godot. I plugin servono dopo, per esigenze specifiche. Per imparare: zero plugin.

Progetti in questa cartella

chirurgo-pasticcione/ — il primo gioco

Primo gioco costruito da zero il 23/07/2026. Il chirurgo maldestro fa cadere gli organi dal tavolo: muovi il **vassoio** con le frecce ← → e prendili al volo. Presa = +1 punto; organo a terra = -1 vita; a zero vite "Operazione Fallita" (INVIO per riprovare). Tutto in un unico script: `main.gd`.

Questa è una **copia di backup**: il progetto "vivo" su cui lavoro è in `C:\Godot\Il mio primo gioco` sul PC.

Idee per i prossimi passi: suoni buffi, un "bisturi" da NON prendere, punteggio record, difficoltà che cresce.

acchiappa-le-stelle/

Mini gioco completo di esempio: muovi una navetta con le frecce e prendi le stelle che cadono. Tutto in un unico script commentato: `main.gd`.

Come aprirlo in Godot (sul tuo PC Windows):

1. Apri **Godot**.
2. In alto scegli la scheda **Progetti** → pulsante **Importa**.
3. Naviga fino a questa cartella `godot/acchiappa-le-stelle/` e seleziona il file `project.godot`.
4. Premi **Importa e modifica**.
5. In alto a destra premi il tasto ► **Esegui** (o `F5`).

File del progetto:

- `project.godot` → configurazione (nome, scena iniziale, dimensione finestra)
- `main.tscn` → la scena principale (un nodo `Node2D` con lo script attaccato)
- `main.gd` → tutto il codice del gioco, commentato in italiano

Riprendiamo da qui — nota per la prossima sessione

Aggiornato: 16/08/2026.

Dove siamo (tutto su `main`)

- **Indice di tutto il materiale:** `02 INDICE.md` in radice — va tenuto aggiornato a ogni aggiunta o cambio. (Documenti tecnici rinumerati `00/01/02` secondo `REGOLE-DOCUMENTI.md`.)
- **Manuale v0.20** — teoria, lettera a mano firmata "Nicola", schede, i 4 capitoli di costruzione degli esercizi 1-2-3-4 (Es4 = vite e game over), progetto boss. Nel Cap. 3 i mattoncini booleano/stato e visible. Ora ha un **Sommario cliccabile** dopo la copertina (+ segnalibri laterali nel PDF) e **tutte** le foto: ogni esercizio ha sia la foto dell'ambiente sia quella del gioco.
- **Super-guida** `CORSO-COMPLETO-v1.3.pdf` — tutti i documenti in un solo PDF, con indice cliccabile e segnalibri (si genera con `manuale/_build/volume_pdf.py`). C'è anche `00 CORSO.md` / `00 CORSO.pdf`, il quadro completo stampabile.
- **Eserciziario v0.15** (PDF `eserciziario-v0.15.pdf` generato) — esercizi 1-4 (4 livelli + foto del risultato) + BOSS, ognuno con la "Guida passo passo — costruiamolo insieme" e la spiegazione del codice riga per riga. **Ogni passo della guida e' stato verificato sulle schermate reali di Godot 4.7.1 in italiano** (vedi nota qui sotto).
- **Esercizi funzionanti** in `esercizi/` (01 bottone, 02 quadrato, 03 moneta). Convenzione nomi: variabile finisce in "Var", nodo in "Scena".
- **Giochi:** `battaglia-navale-3d/` (Affonda la Bonomi) e `gioco-del-quindici/` (15-puzzle con la foto di Serena incorporata, tessere di legno driftwood in rilievo nella vaschetta, sfondo beige; menu solo 3x3/4x4). Entrambi pubblicati sul web (vedi sotto). Aggiunti quattro giochi nuovi: `acchiappa-la-talpa/` (talpe da toccare a tempo, `/talpa/`), `schiva-gli-asteroidi/` (navetta che schiva i sassi, `/asteroidi/`), `rompi-i-mattoni/` (breakout, `/mattoni/`) e `torta-in-faccia/` (tiro alla torta sul bersaglio-faccia, `/torta/`).
Tutti e quattro ora hanno i suoni (presa, splat, rimbalzo/mattone, boom): file WAV in ogni cartella, agganciati via `AudioStreamPlayer` nel codice.

- **Classifica online condivisa** sui quattro giochi arcade (`classifica.gd` , identico in ogni cartella): a fine partita legge/scrive i primi 10 su un **Firestore Realtime Database** via REST. A fine partita appare un overlay con la classifica; se entri nei primi 10 scrivi il nome con una **tastiera a bottoni disegnata nel gioco** (la tastiera di Android nella LineEdit di Godot e `window.prompt` NON erano affidabili → risolto così). Ogni gioco usa una sua chiave (`gioco = "talpa"/"asteroidi"/"mattoni"/"torta"`).
- **Database:** progetto Firestore `esempio1` (id `nicolaregge2` , creato da Nicola nel suo account Google, piano gratuito Spark). URL nel codice: `https://nicolaregge2-default-rtdb.europe-west1.firebaseio.com` . Regole pubbliche permanenti (`.read / .write: true`) → chiunque gioca può scrivere. Claude da qui NON raggiunge il database (proxy lo blocca): il collaudo va fatto sul telefono.
- **Anti-cache:** `/tmp/bust.py` inietta in ogni `docs/*/index.html` una patch che ricarica `index.pck` sempre fresco (`Date.now()` a ogni apertura), così i **link puliti** (senza `?v=`) mostrano da soli l'ultima versione. Da rilanciare dopo ogni export che sovrascrive l'`index.html`.
- **Release v1.0** pubblicata (versione congelata per i ragazzi). La v1.1 è da pubblicare quando Nicola vuole (il tag lo crea lui dal browser, Claude non può).
- Tutte le foto degli esercizi sono in `manuale/immagini/` (es1-ambiente, es1-gioca, es2-ambiente, es2-gioca, es3-ambiente, es3-gioca, es4-ambiente, es4-gioca).

Verifica delle guide sull'interfaccia REALE di Godot (metodo, da ricordare)

Le coordinate e i nomi dei pulsanti nelle guide passo-passo **non si scrivono a memoria**: si verificano guardando l'interfaccia vera. In questo ambiente Claude **puo' far girare Godot 4.7.1 con interfaccia grafica** e fotografarla:

- schermo virtuale **Xvfb** (`:99`), Godot con `--display-driver x11 --rendering-driver opengl3` , screenshot con **Pillow ImageGrab**;
- lingua editor in italiano: in `~/ .config/godot/editor_settings-4.7.tres` , `interface/editor/localization/editor_language = "it"` ;
- i **clic sui pulsanti** (Button) funzionano via XTEST; i clic sugli elementi ad **albero** (Scene tree, file browser) e la **tastiera** NON registrano bene senza window manager → per quei passi si aprono progetti già pronti o si usano schermate note. Le finestre/menu a comparsa vanno bene se sono dialoghi.
- Regola: se non riesco a vedere un passo con i miei occhi, **non lo affermo**: lo scrivo in modo prudente o chiedo a Nicola una schermata. Mai indovinare.

Cose confermate sulle schermate reali (Godot 4.7.1 IT): `Crea / Importa` in alto a SINISTRA nel Gestore progetti; finestra nuovo progetto con "Nome del progetto" in cima, "Percorso del progetto" sotto, Renderer con "Compatibilit ", pulsante "Crea" + casella "Modifica ora" (non "Crea e modifica"); pannello Scena vuoto con i pulsanti radice "Scena 2D / Scena 3D / Interfaccia utente / Altro nodo"; finestra "Crea un nuovo Node" con Cerca/Corrispondenze/Preferiti/Recenti; lo script si attacca con l'icona "Allega uno script" (foglio con + verde) nella barra del pannello Scena; l'import apre una finestra file (scegli `project.godot` → "Apri").

Pubblicazione web dei giochi (GitHub Pages) — COME FUNZIONA

Regola operativa importante (era fonte di confusione, ora   chiara e va ricordata):

- **Sorgente di Pages = "GitHub Actions"** (Settings → Pages → Source). NON "Deploy from a branch". (Prima puntava al ramo vecchio `gioco-battaglia-3d`, perci  i giochi nuovi davano 404: risolto.)

- Un workflow, `.github/workflows/deploy-pages.yml`, pubblica la cartella `docs/` del ramo `main` a ogni push su main (e su avvio manuale).

- **Claude pu  pubblicare/aggiornare da solo** (avvia e segue il workflow da qui):

Nicola non deve toccare impostazioni.

- I giochi web stanno in `docs/<gioco>/`. Link attuali:

- * Gioco del quindici: `https://nicolaregge-pulse.github.io/corso-godot/quindici/`

- * Affonda la Bonomi: `https://nicolaregge-pulse.github.io/corso-godot/`

- Per mettere un gioco sul web: esportarlo da Godot in `docs/<gioco>/` (preset Web, **senza thread**, cos  va anche su iPhone), poi push su main → si pubblica da solo. In questa sessione l'export l'ha fatto Claude scaricando Godot 4.7.1 + i modelli di esportazione nell'ambiente (i modelli non restano tra le sessioni).

Cassaforte "Frau Doktor Bonomi" (app personale di Nicola) — STATO

App web che cifra un file con password (AES-256, tutto sul dispositivo, niente in rete). Stile steampunk/ottone. Sorgente di verit : `docs/cassaforte/index.html` (la lavorazione sta nello scratchpad di sessione, che   effimero). Online:

`https://nicolaregge-pulse.github.io/corso-godot/cassaforte/`.

Il vincolo iPhone (confermato, da NON dimenticare): quando iPhone apre un file "al volo" (anteprima Quick Look di WhatsApp/Mail) **NON esegue il JavaScript**. Perciò il file `[nome].html` che "si apre da solo" funziona su **Android/PC/Mac** ma **NON** in anteprima su iPhone (compare la cassaforte ma il tasto non fa niente: manca proprio l'esecuzione del codice). Il codice gira solo in una **vera pagina web aperta in Safari**.

Strada che su iPhone funziona senza limiti di dimensione (foto comprese):

1. Salvare il file: in WhatsApp/Mail toccare il file → icona **Condividi** in alto a destra → **Salva su File** → **Salva**.

2. Aprire in Safari la pagina Cassaforte → linguetta **Apri** → **Scegli file** → prendere il `[nome].html` salvato → password → **Apri e scarica**.

Il "link con i dati dentro" (un tocco) è stato scartato: ha un limite di dimensione (~40 KB), Nicola vuole **senza limiti**.

Migliorie già pubblicate (PR #56): l'anteprima del file su iPhone ora mostra le istruzioni chiare (Salva su File + link) invece della cassaforte "morta"; aggiunta la rotella con la percentuale mentre prepara il file.

REGOLA (decisa da Nicola): le cose "coi ragazzi" si fanno IN CLASSE

Tutto ciò che riguarda gli studenti — creare gli **account GitHub**, il **primo giro di consegna**, l'**invito alla classe** — si fa **con i ragazzi davanti, a scuola (da settembre), NON prima**. Non prepararle né sollecitarle in anticipo: il materiale è già pronto (`consegne/`), basta usarlo quando la classe c'è.

Cosa manca (prossimi passi)

Con la classe (a settembre, vedi regola sopra): account ragazzi · primo giro di consegna · invito alla classe.

Opzionali quando Nicola vuole (non urgenti, si possono fare anche prima):

- **Release** stabile aggiornata per i ragazzi (tag dal browser) quando si vuole congelare il materiale nuovo (giochi, kit consegne, manuale 0.19).
- Foto `es4-gioca.png` da inserire nel capitolo Esercizio 4 del manuale.
- Ritocchi ai giochi: suono (splat torta, presa talpa) e schermata iniziale.
- Prova Cassaforte su iPhone con Serena (sospesa: da remoto e' scomoda).

Documenti consegnabili anche in PDF (regola di Nicola)

Le **guide/documenti consegnabili** vanno prodotti **anche in PDF**, non solo in MD. Strumento: `manuale/_build/guida_pdf.py` (MD → HTML pulito) + `guida_pdf.js` (HTML → PDF con Chromium). Uso:

```
python3 manuale/_build/guida_pdf.py <guida.md> poi
```

```
node manuale/_build/guida_pdf.js <guida.guida.html> <guida.pdf> .
```

Gia' fatti: `consegne/CREA-ACCOUNT-GITHUB.pdf` , `consegne/INVITO-ALLA-CLASSE.pdf` .

Il manuale e l'eserciziario hanno il loro build dedicato (`genera_pdf.py`).

Kit di consegna dei ragazzi — FATTO (in `consegne/`)

Ogni consegna = una cartella con: `scheda.md` (modulo: cognome, nome, classe, esercizio/titolo, data, "cosa ho fatto con parole mie", "il mio tocco", "dove mi sono bloccato"), il **codice**, le **immagini**.

- Struttura per gestire più classi e più anni:

```
consegne/<anno-classe>/<cognome-nome>/<esercizio>/...
```

- `consegne/_MODELLO/` = kit vuoto che i ragazzi copiano (scheda, ISTRUZIONI con anche "come viene valutata la tua consegna", cartella immagini).

- `consegne/_PROF/` = solo prof: `COME-CORREGGO.md` (giro completo, i 4 segnali, la scala del voto APPROVATA) e `valutazione-MODELLO.md` .

- **Esempio già compilato:** `consegne/2026-2027-1informatica/rossi-mario/es1-bottone/` (scheda + main.gd + screenshot + valutazione) come modello per i ragazzi.

- I **4 segnali** valutati: consegna completa · funziona · il tuo tocco · lo sai spiegare con parole tue. Scala voto CONFERMATA: 9-10 tutto ok e lo sa spiegare; 7-8 funziona e tocco, spiegazione incerta; 6 tocco minimo o spiegazione debole; 5 incompleta o non funziona ma tentativo vero → si può RI-CONSEGNARE. Si parte dall'alto, giudizio sempre incoraggiante, ultima parola al prof.

- Giro: ragazzo consegna → Claude corregge → **voto = numero + breve giudizio** → Claude aggiunge una pagina al **manuale personale** dello studente.

- I ragazzi lavorano con **Gemini**; Claude corregge e compila i loro manuali.

Idee più avanti

- Impaginare il **quaderno-studente** in PDF (base del manuale personale).
- Riordinare gli esercizi per difficoltà; aggiungerne altri.
- Gioco del quindici: eventuale vera texture di legno; Esercizio 4.

Preferenze da ricordare **SEMPRE** (decise da Nicola)

- Aspetto **sobrio, serio, professionale**: niente icone/emoji. Unica eccezione: i **4 livelli d'aiuto** restano colorati (verde/giallo/arancione/rosso).

- **Niente parentesi** nei **titoli** e nelle **scritte in grassetto** (nel testo discorsivo e nel codice possono restare).
- Il libro **parla ai ragazzi** ("tu"), non è una riflessione tra docente e Al.
- **Changelog** solo nel sorgente `.md`, **non** nel PDF. Versione nel footer.
- Ogni PDF ha la **versione nel nome file**; mai riusare una versione.
- **Mai far salvare/consegnare un nome di file che esiste già**: ogni file consegnato ha nome unico con versione; non re-inviare mai un file identico che Nicola ha già (causa "file già esistente" quando salva). Regola sacra.
- Con Nicola e i ragazzi: **tutto visuale** (GitHub Desktop / browser), mai la riga di comando.
- **COORDINATE COMPLETE, SEMPRE (regola vincolante)**. Prima di ogni azione dico, in quest'ordine: 1) **quale APPLICAZIONE** (es. `[APP – GitHub Desktop]`, `[APP – Godot]`, `[BROWSER]`); 2) **quale FINESTRA / SCHEDA**; 3) **quale MENU** → **voce** → **sotto-voce**; 4) **in quale AREA** (pannello a sinistra, barra in alto, ecc.); 5) **l'AZIONE esatta**, una alla volta. Mai un comando "nudo" senza dire DOVE va fatto. Testi/valori da copiare sempre in blocco di codice.
- **Descrizione del commit sempre pronta da me**: ogni volta che so che Nicola dovrà fare un commit (file generati da Godot, modifiche nostre, PDF nuovi), gli fornisco IO il testo del campo Summary in un blocco da copiare, senza che lo debba chiedere. Anticipare, non aspettare.
- **PRECISIONE MICIDIALE nelle guide passo-passo (regola vincolante)**. Alcuni ragazzi "non vanno oltre un centimetro": non colmano i vuoti, fanno ESATTAMENTE ciò che c'è scritto. Ci sono anche ragazzi con disabilità: un termine ambiguo o un passo saltato è un muro, non un fastidio. Quindi ogni guida operativa deve:
 - usare i **nomi esatti come compagno a schermo** (finestre e pulsanti: es. finestra "Crea un nuovo Node", pulsante "Scena 2D");
 - **una micro-azione per riga**, numerata; mai due cose nella stessa frase;
 - **mai** "scegli/vai/sistema" senza dire DOVE e COSA cliccare;
 - **sempre lo stesso termine** per la stessa cosa (niente sinonimi);
 - **guidare lo sguardo** (dire dove guardare prima di cosa fare);
 - meglio ridondanti che ambigui. Scrivere immaginando cosa vede LO STUDENTE, non cosa so io.

02 Indice del materiale del corso

Versione 1.1 — 16/08/2026

Mappa di tutto quello che c'è nel corso, con una riga di descrizione per ogni voce. Si aggiorna a ogni aggiunta o cambiamento.

1. Manuale, cartella manuale

1. Libro di testo: `manuale/manuale.md` , versione 0.20. PDF pronto `manuale/manuale-v0.20.pdf` . La teoria di Godot, spiegata ai ragazzi, con il sommario cliccabile, le foto dell'ambiente e del gioco di ogni esercizio 1-4 e i capitoli di costruzione.
2. Eserciziario: `manuale/eserciziario.md` , versione 0.15. PDF pronto `manuale/eserciziario-v0.15.pdf` . Gli esercizi coi quattro livelli di aiuto, la guida passo passo e la spiegazione del codice riga per riga.
3. Quaderno dello studente: `manuale/quaderno-studente-TEMPLATE.md` . Il modello del manuale personale che ogni ragazzo fa crescere lezione dopo lezione.
4. Immagini: `manuale/immagini/` . Gli screenshot richiamati nel libro.

2. Esercizi funzionanti, cartella esercizi

1. Progetti Godot già pronti, uno per cartella. Convenzione dei nomi: la variabile finisce in Var, il nodo nella scena finisce in Scena.
2. Cartelle:
3. `esercizi/01-bottone-che-saluta/` .
4. `esercizi/02-muovi-il-quadrato/` .
5. `esercizi/03-prendi-la-moneta/` .
6. `esercizi/04-acchiappa-le-stelle/` .

3. Giochi

1. Giocabili subito dal telefono o dal browser, anche a computer spento. I quattro giochi arcade (talpa, asteroidi, mattoni, torta) hanno i suoni e la classifica online condivisa dei primi 10 giocatori: a fine partita si scrive il nome con una tastiera a bottoni e il punteggio va in una classifica unica per tutti, su un database Firebase.
2. Indirizzi:

3. Affonda la Bonomi, battaglia navale 3D.

<https://nicolaregge-pulse.github.io/corso-godot/>

2. Gioco del Quindici, con la foto incorporata e le tessere di legno.

<https://nicolaregge-pulse.github.io/corso-godot/quindici/>

3. Acchiappa la talpa.

<https://nicolaregge-pulse.github.io/corso-godot/talpa/>

4. Schiva gli asteroidi.

<https://nicolaregge-pulse.github.io/corso-godot/asteroidi/>

5. Rompi i mattoni.

<https://nicolaregge-pulse.github.io/corso-godot/mattoni/>

6. Torta in faccia.

<https://nicolaregge-pulse.github.io/corso-godot/torta/>

3. Sorgenti Godot dei giochi, in radice:

1. [battaglia-navale-3d/](#) , Affonda la Bonomi.
2. [gioco-del-quindici/](#) , il 15 puzzle.
3. [acchiappa-la-talpa/](#) , le talpe da toccare a tempo.
4. [schiva-gli-asteroidi/](#) , la navetta che schiva i sassi.
5. [rompi-i-mattoni/](#) , racchetta, pallina e muro di mattoni.
6. [torta-in-faccia/](#) , tiro alla torta su un bersaglio a cerchi sulla faccia.
7. [chirurgo-pasticcione/](#) , primo gioco, il vassoio che prende gli organi.
8. [acchiappa-le-stelle/](#) , mini gioco di esempio, la navetta che prende le stelle.

4. Cassaforte, app personale

1. App che chiude un file con una password, tutto sul dispositivo. Online:

<https://nicolaregge-pulse.github.io/corso-godot/cassaforte/>

2. Sorgente pubblicata: [docs/cassaforte/index.html](#) .

5. Kit di consegna dei ragazzi, cartella consegne

1. Tutto per raccogliere e correggere i lavori dei ragazzi.
2. Contenuto:
3. [consegne/CREA-ACCOUNT-GITHUB.md](#) , come i ragazzi si fanno l'account GitHub, si fa in classe il primo giorno.

4. `consegne/INVITO-ALLA-CLASSE.md` , la guida del primo giro, dal fork alla prima consegna, tutta da browser.
5. `consegne/_MODELLO/` , il kit vuoto che i ragazzi copiano: `scheda.md` e `ISTRUZIONI.md` , che spiega anche come vengono valutati.
6. `consegne/_PROF/` , solo prof: `COME-CORREGGO.md` con i quattro segnali e la scala del voto, e `valutazione-MODELLO.md` .
7. `consegne/2026-2027-1informatica/rossi-mario/es1-bottone/` , un esempio già compilato, da far vedere alla classe.

6. Versione consegnata ai ragazzi

1. Release v1.0, la versione congelata e stabile pubblicata per i ragazzi.

7. Documenti tecnici e di servizio

1. Documento 00 Corso, il quadro completo di tutto quello che abbiamo creato, in un unico riepilogo stampabile.
2. Documento 01 Glossario, i termini e le sigle del corso spiegati.
3. Documento 03 Scaletta lezioni, il piano delle prime lezioni della Fase 1: cosa spiegare, in che ordine, con quale esercizio, con la piccola vittoria da mostrare a ogni lezione.
4. Documento 04 Struttura del repository, l'albero del repository: com'è organizzato adesso e dove si arriverà con la Fase 2.
5. `REGOLE-DOCUMENTI.md` , le regole di formattazione dei documenti del corso.
6. `CORSO-COMPLETO-v1.3.pdf` , la super-guida: tutti i documenti, con le immagini, in un solo PDF, con Sommario e segnalibri cliccabili. Si genera con `manuale/_build/volume_pdf.py` .
7. `README.md` , la porta d'ingresso del repository, rimanda a questo indice.
8. `CLAUDE.md` , preferenze e contesto del corso.
9. Nota di servizio interna RIPRENDIAMO-DA-QUI, dove siamo e cosa manca tra una sessione e l'altra.
10. `docs/` , la cartella che viene pubblicata sul web da GitHub Pages.
11. `.github/workflows/deploy-pages.yml` , pubblica `docs/` sul web a ogni modifica.